

Classical ML Algorithms II: Interview, Mathematics, and Implementation Guide

This guide covers boosting, clustering, dimensionality reduction, association mining, anomaly detection, and latent-factor methods. Every chapter is self-contained and follows the same interview-oriented structure. Code targets Python 3 with scikit-learn; optional libraries are identified explicitly.

Notation: n = samples, d = features, K = clusters/components, T = trees/iterations, and m = observed user-item interactions. Fit every learned preprocessing step on training data only.

AdaBoost

1. Overview

AdaBoost (Adaptive Boosting) converts many weak learners into a strong supervised model. Learners are trained sequentially; examples misclassified by the current ensemble receive more influence in the next round. The classic weak learner is a decision stump (a depth-1 tree).

AdaBoost is useful for small or medium structured datasets, interpretable boosting demonstrations, face detection cascades, and interview questions about margins and sample reweighting. Modern gradient-boosted trees usually win on large tabular benchmarks, but AdaBoost remains foundational.

2. Intuition

Imagine a tutor repeatedly testing a class. After every test, the tutor spends more time on questions students missed. Each new tutor is only slightly better than guessing, but the final answer is a weighted vote: tutors with lower error get more authority.

For example, the first stump may separate loan applicants by debt ratio. The next focuses on applicants that rule misclassified and may split on income. Their weighted votes create a nonlinear classifier.

3. Prerequisites

- Binary classification and labels encoded as -1 and $+1$
- Decision stumps and weighted classification error
- Exponential and logarithmic functions
- Ensemble voting, margins, overfitting, and train/test splitting
- Basic scikit-learn pipelines and classification metrics

4. Core Concepts

Concept	Meaning and importance	Simple example	Common interview angle
Weak learner	Model performing slightly better than chance	One decision stump	Why weak learners can form a strong learner
Sample weights	Distribution over training rows	A missed row gets higher weight	How the next learner focuses on errors
Learner weight	Vote strength based on weighted error	10% error gets more vote than 40%	Derive α_t
Margin	$yF(x)$; signed confidence of ensemble	Positive means correct side	AdaBoost often improves margins
Exponential loss	Smooth upper bound on classification error	Penalizes confident wrong answers heavily	Relation between boosting and loss minimization
Sequential dependence	Round t depends on previous errors	Trees cannot all train independently	AdaBoost vs bagging

5. Algorithm / Working Process

Input: training pairs (x_i, y_i) , $y_i \in \{-1, +1\}$, number of rounds T , and a weak learner.

1. Initialize each sample weight as $w_i = 1/n$.
2. Fit weak learner $h_t(x)$ using the current weights.
3. Compute weighted error ϵ_t .
4. Give the learner vote weight α_t ; a more accurate learner gets a larger positive value.
5. Multiply weights of misclassified samples and reduce weights of correctly classified samples.
6. Normalize weights so they sum to one.
7. Repeat and return the sign of the weighted vote.

At inference, all trained learners run; classification uses $\text{sign}(\sum_t \alpha_t h_t(x))$. For probability output, libraries transform the ensemble score, but calibration should be checked separately.

6. Mathematical Foundation

Weighted error:

$$\epsilon_t = \frac{\sum_{i=1}^n w_i \mathbf{1}[y_i \neq h_t(x_i)]}{\sum_{i=1}^n w_i}.$$

Learner weight for binary AdaBoost:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right).$$

Sample-weight update:

$$w_i \leftarrow \frac{w_i \exp(-\alpha_t y_i h_t(x_i))}{Z_t},$$

where z_t normalizes the weights. A mistake has $y_i h_t(x_i) = -1$, so its weight is multiplied by $\exp(\alpha_t)$.

The score and prediction are

$$F_T(x) = \sum_{t=1}^T \alpha_t h_t(x), \quad \hat{y} = \text{sign}(F_T(x)).$$

AdaBoost greedily minimizes exponential loss $\sum_i \exp(-y_i F(x_i))$. If $\epsilon_t = 0.5$, then $\alpha_t = 0$; a random learner adds nothing. If error exceeds 0.5 , its prediction can be inverted in the binary case.

7. Practical Implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import classification_report, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

stump = DecisionTreeClassifier(max_depth=1, random_state=42)
model = AdaBoostClassifier(
    estimator=stump,
    n_estimators=200,
    learning_rate=0.05,
    random_state=42,
)
model.fit(X_train, y_train)

pred = model.predict(X_test)
prob = model.predict_proba(X_test)[:, 1]
print(classification_report(y_test, pred))
print("ROC-AUC:", roc_auc_score(y_test, prob))
```

8. Code Explanation

The split is stratified to preserve class ratios. A stump deliberately has high bias and acts as a weak learner. `n_estimators` controls boosting rounds; `learning_rate` shrinks each contribution. The test set is untouched during fitting. ROC-AUC evaluates ranking, while the classification report exposes class-specific precision and recall.

9. Training / Evaluation

Use a validation set or cross-validation to tune `n_estimators`, `learning_rate`, and base-tree complexity together. For imbalance, inspect PR-AUC, recall, F1, and business cost rather than accuracy alone. Plot training and validation scores by round; a widening gap signals overfitting. Scaling is usually unnecessary for tree stumps, but needed if the weak learner is distance- or margin-based. AdaBoost is sensitive to bad labels, so investigate repeatedly high-weight samples.

10. Complexity and Cost

With stumps and sorted feature search, a rough training cost is $O(T \cdot n \cdot d \cdot \log n)$; exact cost depends on the tree implementation. Inference is $O(Td)$ for stumps and memory is proportional to all stored trees. Training rounds are sequential, although feature/split work may be parallelized internally. CPU is sufficient for typical use.

11. Common Use Cases

- Binary classification on clean structured data
- Viola-Jones-style face detection cascades
- Churn, response, and risk baselines
- Feature ranking with shallow-tree ensembles
- Teaching boosting, sample weighting, and margins

12. Common Mistakes

- Using deep trees and losing the weak-learner regularization
- Ignoring mislabeled points that accumulate extreme weight
- Tuning on the test set or preprocessing before the split
- Reporting accuracy on severe imbalance
- Assuming `predict_proba` is automatically calibrated
- Confusing sample weights with resampling or class weights

13. Edge Cases / Limitations

AdaBoost can chase label noise and outliers because hard cases repeatedly gain weight. It is sequential, less parallelizable than bagging, and generally less capable than XGBoost/LightGBM/CatBoost on large mixed-type data. A base learner with error near random contributes little. It also does not natively solve missing-data or high-cardinality categorical preprocessing.

14. Variations

Variation	What changes	When to use	Importance
Discrete AdaBoost/SAMME	Uses predicted classes; supports multiclass	Multiclass weak learners	Placement: high
Real AdaBoost/SAMME.R	Uses class confidence/probability	Better weak-learner confidence information	Placement: medium; version-dependent API

Variation	What changes	When to use	Importance
GentleBoost	Uses gentler regression-style updates	Noisy observations	Research/project: medium
AdaBoost.R2	Reweights regression errors	Regression with clean targets	Placement: medium
Cost-sensitive AdaBoost	Adds unequal mistake costs	Fraud/medical recall	Project: medium

15. Related Topics

AdaBoost changes **row importance** from mistakes; gradient boosting fits the **negative gradient of a loss**. Random Forest trains trees independently on bootstraps and random features, reducing variance; AdaBoost trains sequentially and often reduces bias. XGBoost adds second-order optimization and explicit tree regularization. Exponential loss is less robust to outliers than logistic loss.

16. Interview Questions

1. **What makes AdaBoost adaptive?** Sample influence changes after every round according to current mistakes.
2. **Why use stumps?** They are cheap, high-bias weak learners whose diverse corrections combine well.
3. **What if weighted error is 0.5?** The learner receives zero vote weight.
4. **What if error is zero?** The theoretical vote becomes infinite; implementations stop or clip numerically.
5. **Why is it sensitive to label noise?** Mislabeled samples remain difficult and repeatedly gain weight.
6. **AdaBoost vs bagging?** AdaBoost is sequential and reweights errors; bagging trains resampled models independently and averages them.
7. **What loss does AdaBoost minimize?** Exponential loss in the ensemble margin.
8. **Does AdaBoost require scaling?** Not for tree learners; possibly for other base estimators.
9. **How does multiclass AdaBoost work?** SAMME adjusts the learner vote using class count and weighted error.
10. **Can it regress?** Yes, AdaBoost.R2 adapts weighting to continuous errors.
11. **How do learning rate and estimator count interact?** Smaller steps generally require more rounds and can improve generalization.
12. **When would you reject AdaBoost?** Heavy label noise, huge data, complex categories, or when modern GBDT libraries clearly validate better.

17. Practice Tasks

- Implement binary AdaBoost with NumPy and decision stumps.
- Plot sample weights across rounds and inspect the hardest observations.
- Compare a stump, AdaBoost, and Random Forest on breast-cancer data.
- Corrupt 10% of labels and analyze robustness.
- Add threshold tuning for a recall-constrained medical task.

18. Project Ideas

Project	What it does	Tech stack	Dataset suggestion	Resume value
Explainable Churn Booster	Predicts churn and visualizes hard customers by round	pandas, sklearn, SHAP/permutation importance	Telco Churn	Ensembles + explainability
Face Cascade Study	Reproduces weak-feature boosting ideas	OpenCV, sklearn	MIT CBCL face data	Classical CV history
Noise-Robustness Benchmark	Compares AdaBoost with RF/GBDT under label corruption	sklearn, matplotlib	UCI datasets	Experimental rigor

19. Quick Revision

- **Key idea:** sequentially focus on mistakes and combine weighted weak learners.
- **Main formula:** $\alpha_t = 0.5 \log((1-\text{error})/\text{error})$.
- **When to use:** clean, modest tabular classification and interpretable boosting baselines.
- **Important metrics:** ROC-AUC/PR-AUC, F1, recall, calibration.
- **Common traps:** noisy labels, deep base trees, test-set tuning.
- **Interview one-liner:** "AdaBoost greedily minimizes exponential loss by reweighting hard examples."

20. Final Cheat Sheet

Item	Summary
Definition	Adaptive weighted ensemble of sequential weak learners
Input/output	Labeled feature matrix -> class score/probability
Main steps	Weight rows, fit learner, compute error/vote, update weights, vote
Key hyperparameters	<code>n_estimators</code> , <code>learning_rate</code> , base-estimator depth
Metrics	F1, ROC-AUC, PR-AUC, recall, log loss
Pros	Simple, effective, interpretable mechanism
Cons	Noise-sensitive, sequential, weaker modern tabular performance
Best use cases	Clean small/medium classification; boosting education

Gradient Boosting

1. Overview

Gradient Boosting builds an additive model one weak learner at a time. Each new learner approximates the direction that most reduces a differentiable loss. Trees are common, but the core idea is functional gradient descent, not merely "fit residuals."

It is used for credit scoring, conversion prediction, insurance, forecasting features, ranking, and strong tabular baselines. It supports regression, classification, ranking, and custom differentiable objectives.

2. Intuition

Begin with one constant prediction. Ask, "In what direction should each prediction move to reduce the loss?" Fit a small tree to those desired corrections and add a shrunk version. Repeating many small corrections yields a flexible function.

With squared error, desired corrections are ordinary residuals. With logistic loss, they are probability-aware pseudo-residuals.

3. Prerequisites

- Decision trees, residuals, gradients, and differentiable losses
- Additive models and learning rates
- Bias-variance trade-off and regularization
- Logits, sigmoid, MSE, and cross-entropy
- Cross-validation, early stopping, and leakage-safe pipelines

4. Core Concepts

Concept	Meaning and importance	Example	Interview angle
Additive model	Sum of weak learners	$F = \text{constant} + \text{small trees}$	Why training is sequential
Pseudo-residual	Negative loss derivative w.r.t. score	$y - F$ for squared loss	Residuals vs general gradients
Shrinkage	Scale each new tree by η	$\eta = 0.05$	Learning-rate/tree-count trade-off
Tree depth	Interaction order/capacity	Depth 1 models mostly additive effects	Main overfitting control
Stochastic boosting	Subsample rows/features	Train each tree on 80% rows	Regularization and diversity
Stage-wise fitting	Earlier trees remain fixed	Add, do not refit, previous trees	Functional gradient descent

5. Algorithm / Working Process

Input: (X, y) , differentiable loss L , weak-learner family, rounds T , step size η .

1. Choose the best constant model F_0 , such as mean target for MSE or log-odds for binary log loss.
2. At round t , compute pseudo-residuals $r_{it} = -dL(y_i, F(x_i)) / dF(x_i)$.
3. Fit a shallow regression tree h_t to (X, r_t) .
4. Optionally solve for an optimal leaf/line-search multiplier.

5. Update $F_t(x) = F_{t-1}(x) + \eta h_t(x)$.
6. Stop by round limit or validation early stopping.

Inference sums all tree outputs and transforms scores when needed: sigmoid for binary classification and softmax for multiclass.

6. Mathematical Foundation

The ensemble is

$$F_T(x) = F_0(x) + \eta \sum_{t=1}^T \rho_t h_t(x).$$

At stage t , pseudo-residuals are

$$r_{it} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{t-1}}.$$

For squared loss $L=0.5(y-F)^2$, $r=y-F$. For binary log loss with $p=\sigma(F)$, $r=y-p$. A line search may choose

$$\rho_t = \arg \min_{\rho} \sum_i L(y_i, F_{t-1}(x_i) + \rho h_t(x_i)).$$

Small η regularizes by preventing one tree from making a large correction. Stochastic row subsampling reduces correlation and variance but changes the gradient estimate.

7. Practical Implementation

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

model = GradientBoostingClassifier(
    n_estimators=300,
    learning_rate=0.03,
    max_depth=2,
    subsample=0.8,
    validation_fraction=0.15,
    n_iter_no_change=20,
    random_state=42,
)
model.fit(X_train, y_train)
prob = model.predict_proba(X_test)[: , 1]
print("Trees used:", model.n_estimators_)
```

```
print("ROC-AUC:", roc_auc_score(y_test, prob))
print("Log loss:", log_loss(y_test, prob))
```

8. Code Explanation

Shallow depth-2 trees limit interactions. `subsample=0.8` creates stochastic gradient boosting. The model reserves part of the training set for early stopping and never uses the test set. `n_iter_no_change` stops after validation improvement stalls. AUC checks ranking; log loss checks probability quality and punishes confident mistakes.

9. Training / Evaluation

Tune learning rate and estimator count jointly, then tree depth/leaf size and subsampling. Use stratified CV for iid classification, grouped CV for entities, and forward-chaining for time data. For regression choose MAE/Huber when outliers matter; for imbalance prefer PR-AUC and threshold-aware costs. Calibration can deteriorate even with good ranking. Compare against a dummy model, linear baseline, Random Forest, and a modern GBDT.

10. Complexity and Cost

Approximate training cost is $O(T \cdot n \cdot d \cdot \log n)$ for conventional trees; histogram variants can be much faster. Inference traverses every tree: roughly $O(T \cdot \text{depth})$ per row. Memory is the sum of tree nodes. Rounds are sequential, limiting tree-level parallelism. CPU is usually enough; GPUs matter primarily in optimized libraries.

11. Common Use Cases

- Credit, fraud, churn, and conversion scoring
- House-price and demand regression
- Learning-to-rank variants
- Nonlinear tabular baselines
- Mixed feature interactions after safe encoding

12. Common Mistakes

- Saying every next tree fits residuals without qualifying the loss
- Combining high depth, high learning rate, and many trees
- Selecting early stopping rounds on the test set
- Leakage through target encoding or time-unaware splits
- Reading impurity importance as causal evidence
- Ignoring probability calibration or threshold selection

13. Edge Cases / Limitations

Boosting can chase noisy labels, extrapolates poorly outside the feature range, and trains sequentially. Classic implementations may require imputation and categorical encoding. Very high-dimensional sparse linear problems can favor linear models. Tiny data may not support extensive tuning, while smooth physical relationships may be better served by models with known structure.

14. Variations

Variation	What changes	When to use	Importance
Stochastic GBM	Samples rows per tree	Reduce variance and cost	Placement: high
Histogram GBM	Bins continuous values	Large data; native missing handling	Placement/project: high
Robust-loss GBM	Huber/absolute/quantile loss	Outliers or prediction intervals	Placement: high
XGBoost	Second-order regularized trees	General high-performance tabular work	Essential
LightGBM	Histogram, leaf-wise growth	Very large/high-dimensional tables	Essential
CatBoost	Ordered categorical statistics	Category-heavy tables	Essential

15. Related Topics

Random Forest averages independent deep trees to reduce variance; gradient boosting adds shallow trees sequentially to reduce loss/bias. AdaBoost changes example weights under exponential loss. XGBoost, LightGBM, and CatBoost implement engineered and regularized forms. Gradient descent optimizes finite parameters; gradient boosting performs analogous descent over functions.

16. Interview Questions

- What does the next tree predict?** The negative derivative of loss with respect to the current prediction score.
- When are pseudo-residuals ordinary residuals?** Under squared-error loss.
- Why shallow trees?** They restrict interaction complexity and make each learner a small correction.
- Why does a lower learning rate need more trees?** Each stage moves a shorter distance in function space.
- Bagging vs boosting?** Independent averaging for variance reduction vs sequential loss correction.
- Can boosting optimize classification loss?** Yes; it fits gradients of log loss and works in logit space.
- What is stochastic boosting?** Using row/feature subsamples during stages for regularization and speed.
- Why early stopping?** It selects effective ensemble size before validation performance worsens.
- Can trees train in parallel across rounds?** Not generally, because each round needs prior predictions.
- How does depth relate to interactions?** A depth- d tree can model interactions involving up to roughly d sequential splits.
- Why no feature scaling for tree GBM?** Order-based tree splits are invariant to monotonic rescaling.
- Why can probabilities be poor despite high AUC?** Ranking and calibration are different properties.

17. Practice Tasks

- Implement squared-loss boosting with NumPy and shallow regression trees.

- Plot train/validation loss against boosting rounds.
- Compare depths 1, 2, and 5 while holding leaf count in view.
- Replace squared loss with Huber loss on contaminated regression data.
- Diagnose time leakage in a customer-event dataset.

18. Project Ideas

Project	What it does	Tech stack	Dataset suggestion	Resume value
Claim Severity Model	Predicts insurance claim cost with robust losses	pandas, sklearn	Kaggle Allstate subset	Regression + loss design
Propensity Scoring Lab	Predicts campaign response and calibrates thresholds	sklearn, matplotlib	UCI Bank Marketing	End-to-end decision ML
Boosting Diagnostics	Visualizes gradients, stages, bias and variance	NumPy, sklearn, Streamlit	Synthetic + UCI	Strong conceptual depth

19. Quick Revision

- **Key idea:** fit weak learners to negative loss gradients.
- **Main formula:** $F_t = F_{(t-1)} + \eta h_t$.
- **When to use:** nonlinear supervised tabular prediction.
- **Important metrics:** task metric plus log loss/calibration.
- **Common traps:** test-set early stopping, excessive depth/rate, leakage.
- **Interview one-liner:** "Gradient boosting is stage-wise functional gradient descent, usually with trees."

20. Final Cheat Sheet

Item	Summary
Definition	Sequential additive learner minimizing differentiable loss
Input/output	Labeled features -> score/value/probability
Main steps	Initialize, compute gradients, fit tree, shrink/add, repeat
Key hyperparameters	Trees, learning rate, depth/leaves, subsample, loss
Metrics	AUC/F1/log loss or RMSE/MAE; calibration
Pros	Accurate, nonlinear, interactions, flexible objectives
Cons	Sequential, tunable, noise-sensitive, weak extrapolation
Best use cases	Structured supervised learning

XGBoost

1. Overview

XGBoost is an optimized, regularized gradient-tree boosting library. It is a standard choice for tabular competitions and production models because of strong accuracy, missing-value handling, and mature tooling.

2. Intuition

It is gradient boosting with more discipline: each new tree corrects errors, while explicit penalties discourage overly complex trees and efficient engineering makes the process fast.

3. Prerequisites

Gradient boosting, decision trees, derivatives, regularization, cross-validation, and categorical feature encoding.

4. Core Concepts

- **Second-order objective:** uses gradient and Hessian; gives better split approximation. Interview: it optimizes a Taylor expansion of loss.
- **Regularization:** penalizes leaf count and leaf weights; limits complexity.
- **Column/row sampling:** adds diversity and speed; reduces overfit.

5. Algorithm / Working Process

For each round, compute first/second loss derivatives, score candidate splits using their aggregate derivatives, grow a regularized tree, then add its weighted output to current logits. Missing values follow learned default split directions.

6. Mathematical Foundation

Objective: $\sum l(y_i, \hat{y}_i) + \sum \Omega(f_t)$, commonly $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$. For a leaf with gradient sum G and Hessian sum H , optimal weight is approximately $-G/(H+\lambda)$; split gain compares child versus parent scores.

7. Practical Implementation

```
# pip install xgboost
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
model = XGBClassifier(n_estimators=500, learning_rate=.03, max_depth=4,
                      subsample=.8, colsample_bytree=.8,
                      reg_lambda=1.0, eval_metric="logloss", random_state=42)
model.fit(X_train, y_train)
prob = model.predict_proba(X_test)[:, 1]
```

```
print("ROC-AUC:", roc_auc_score(y_test, prob))
print("Log loss:", log_loss(y_test, prob))
```

8. Code Explanation

Use low `learning_rate` with many estimators. `subsample` and `colsample_bytree` regularize; `eval_metric` makes optimization reporting explicit. In real work add an evaluation set and early stopping.

9. Training / Evaluation

Tune depth/min-child-weight, learning rate/estimators, sampling, `reg_lambda`, and `reg_alpha`. Use early stopping on validation data and native categorical support only when version/data pipeline supports it; otherwise encode safely.

10. Complexity and Cost

More optimized than basic boosting; histogram/parallel split search helps. Training remains sequential by rounds; memory and CPU/GPU usage grow with rows, bins, and trees.

11. Common Use Cases

Credit risk, fraud, click prediction, ranking, pricing, and Kaggle-style tabular problems.

12. Common Mistakes

Using test set for early stopping, setting both deep trees and high rate, ignoring imbalance (`scale_pos_weight`), one-hot exploding cardinal categories, and skipping reproducible seeds.

13. Edge Cases / Limitations

Can overfit noisy labels, needs tuning, and may be excessive for a simple linear relationship. Tree outputs do not extrapolate smoothly beyond training patterns.

14. Variations

- **gbtree**: default tree booster; essential.
- **gblinear**: linear booster; use for sparse linear relationships.
- **DART**: dropout trees; can regularize but is slower/less routine.

15. Related Topics

XGBoost, LightGBM, and CatBoost are gradient boosting libraries. LightGBM prioritizes histogram/leaf-wise speed; CatBoost excels with categorical features.

16. Interview Questions

Question	Answer
XGBoost over classic GBM?	Regularization, second-order splits, optimized/parallel implementation, missing handling.
What is Hessian?	Second derivative of loss used in split/leaf approximation.
What is gamma?	Minimum loss reduction required to split.
Lambda/alpha?	L2/L1 regularization on leaf weights.
Why early stop?	Selects tree count by validation performance.
Handle imbalance?	Weights, PR metrics, threshold, resampling.
Why low learning rate?	Smaller safer updates, usually with more trees.
Missing values?	Learns a default direction per split.
Feature importance caveat?	Gain/count are not causal and can be biased.
XGBoost vs RF?	Sequential bias reduction versus independent variance reduction.

17. Practice Tasks

Train with early stopping; tune `max_depth` and `min_child_weight`; compare class weights; inspect SHAP/permutation importance; compare CPU and histogram methods.

18. Project Ideas

- **Default probability model:** XGBoost + calibration, Home Credit subset; high resume relevance.
- **Fraud ranking system:** XGBoost + PR-AUC, credit-card fraud; shows imbalance handling.
- **Demand classification:** tabular retail features; showcase time-safe validation.

19. Quick Revision

Key idea: regularized second-order tree boosting. Formula: $\text{loss} + \gamma T + \lambda \sum w^2 / 2$. Use: high-performing tabular. Metrics: AUC/PR-AUC/F1. Trap: validation leakage. One-liner: "Industrial-strength regularized gradient boosting."

20. Final Cheat Sheet

Definition: optimized gradient-tree ensemble. Input/output: table $\rightarrow$ logits/probabilities. Steps: derivatives, split gain, additive trees. Hyperparameters: rate, trees, depth, child weight, subsample, regularization. Pros: strong/fast; cons: tuning. Best: tabular benchmarks.

LightGBM

1. Overview

LightGBM is a histogram-based gradient boosting library designed for high speed and large datasets. It often trains faster than level-wise boosting on large tabular data.

2. Intuition

Instead of checking every exact threshold, bucket feature values into bins. Grow the currently most promising leaf first, spending model capacity where error reduction is largest.

3. Prerequisites

Gradient boosting, histograms/bins, tree growth strategies, regularization, and categorical encoding.

4. Core Concepts

- **Histogram splitting:** discretizes values; faster, lower memory. Interview: bins approximate split thresholds.
- **Leaf-wise growth:** split the leaf with best gain, not every level; accurate but can overfit.
- **GOSS/EFB:** gradient-based row sampling and exclusive-feature bundling; advanced interview topics for speed.

5. Algorithm / Working Process

Bin features, calculate gradient statistics, choose the best leaf/split by gain, grow leaf-wise under leaf/depth constraints, and add trees sequentially. It supports categorical features via specialized splits when supplied correctly.

6. Mathematical Foundation

It minimizes the same regularized boosting objective as XGBoost but aggregates gradients/Hessians per histogram bin. Leaf-wise selection greedily chooses the split with greatest estimated loss reduction globally.

7. Practical Implementation

```
# pip install lightgbm
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import train_test_split
from lightgbm import LGBMClassifier

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
model = LGBMClassifier(n_estimators=500, learning_rate=.03, num_leaves=31,
                       min_child_samples=20, subsample=.8,
                       colsample_bytree=.8, random_state=42, verbosity=-1)
model.fit(X_train, y_train)
prob = model.predict_proba(X_test)[:, 1]
```

```
print("ROC-AUC:", roc_auc_score(y_test, prob))  
print("Log loss:", log_loss(y_test, prob))
```

8. Code Explanation

`num_leaves` is the central complexity control for leaf-wise trees. Keep learning rate low and use validation callbacks/early stopping in production training.

9. Training / Evaluation

Tune `num_leaves`, `min_child_samples`, `max_depth`, learning rate, feature/bagging fractions. Guard against overfit with leaf limits and early stopping; use time-aware splits for events.

10. Complexity and Cost

Histogram bins reduce split-search cost and memory. It is CPU-efficient and GPU-capable; leaf-wise growth can create deeper, more costly individual paths.

11. Common Use Cases

Large clickstream datasets, ranking, ad-tech, credit models, and high-row-count tabular systems.

12. Common Mistakes

Unbounded leaf-wise trees, treating integer category IDs as ordered numbers, using too many bins needlessly, test-set early stopping, and defaulting to accuracy on imbalance.

13. Edge Cases / Limitations

Small datasets can overfit; categorical handling still requires correct dtypes/validation. Leaf-wise trees can be less interpretable and reproducibility may need controlled threads/seeds.

14. Variations

- **DART booster:** dropout boosting; use when standard boosting overfits.
- **GOSS:** samples by gradient magnitude; important concept for large data.
- **Ranker:** ranking objective; valuable recommender/search extension.

15. Related Topics

Compared with XGBoost it uses histogram and leaf-wise growth; CatBoost often needs less categorical preprocessing. All are gradient-boosted decision trees.

16. Interview Questions

Question	Answer
Leaf-wise vs level-wise?	Leaf-wise splits best-gain leaf; level-wise expands a depth level uniformly.
Why histogram bins?	Faster approximate split search with lower memory.

Question	Answer
Main overfit control?	<code>num_leaves</code> , minimum child samples, depth, early stopping.
What is GOSS?	Keeps high-gradient rows and samples low-gradient rows.
What is EFB?	Bundles rarely co-occurring sparse features.
Why fast?	Histograms, efficient implementation, sampling/bundling.
Handle categories?	Mark/use categorical features correctly; do not impose fake order.
LightGBM vs XGBoost?	Often faster/leaf-wise; XGBoost is level-wise by default and mature.
Why small learning rate?	Regularizes additive updates.
When avoid?	Very small/noisy data or when simple interpretable model suffices.

17. Practice Tasks

Benchmark LightGBM and XGBoost; tune leaves versus depth; compare categorical encoding methods; use early stopping; test GOSS on a large dataset.

18. Project Ideas

- **Click-through prediction:** LightGBM, Criteo sample; large-scale tabular story.
- **Loan-default model:** native categories + calibration, LendingClub subset.
- **Search ranker:** LGBMRanker, Microsoft LETOR; advanced recommendation/search project.

19. Quick Revision

Key idea: fast histogram, leaf-wise boosting. Formula: regularized gradient objective. Use: large tabular data. Metrics: AUC/NDCG/F1. Trap: too many leaves. One-liner: "A fast leaf-wise histogram GBDT."

20. Final Cheat Sheet

Definition: efficient gradient-boosted trees. Input/output: table → scores/classes. Steps: bin, gradients, best-leaf split, add tree. Hyperparameters: leaves, child samples, rate, trees. Pros: fast; cons: overfit risk. Best: high-volume tables.

CatBoost

1. Overview

CatBoost is gradient boosting designed to handle categorical variables and reduce target leakage through ordered statistics. It is an excellent tabular option when categories are prominent.

2. Intuition

Rather than naïvely replacing a category with its full-dataset target average, it encodes each row using only earlier rows in a random ordering—like predicting without peeking at that row's answer.

3. Prerequisites

Gradient boosting, categorical variables, target encoding leakage, ordered statistics, cross-validation, and log loss.

4. Core Concepts

- **Ordered target statistics:** category encodings use prior rows only; prevents target leakage. Interview: random permutations make this usable for training.
- **Ordered boosting:** reduces prediction shift between train and inference.
- **Symmetric trees:** same split condition per depth; efficient and regularized.

5. Algorithm / Working Process

Pass categorical column indices; CatBoost creates ordered category statistics under permutations, grows symmetric trees using gradients, and aggregates tree scores into probabilities.

6. Mathematical Foundation

For category value c , an ordered encoding resembles $(\sum \text{prior targets for } c + a \cdot \text{prior}) / (\text{count prior } c + a)$. The ensemble still minimizes regularized classification loss through boosting; order avoids using a row's own label in its encoding.

7. Practical Implementation

```
# pip install catboost
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import train_test_split
from catboost import CatBoostClassifier

data = load_breast_cancer(as_frame=True)
X, y = data.data.copy(), data.target
# A fixed-bin string feature demonstrates native categorical handling.
X["radius_band"] = pd.cut(
    X["mean radius"], bins=[-np.inf, 12, 16, np.inf],
    labels=["small", "medium", "large"]
).astype(str)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
model = CatBoostClassifier(iterations=500, depth=6, learning_rate=.05,
                           loss_function="Logloss", verbose=False, random_seed=42)
model.fit(X_train, y_train, cat_features=["radius_band"])
prob = model.predict_proba(X_test)[:, 1]
print("ROC-AUC:", roc_auc_score(y_test, prob))
print("Log loss:", log_loss(y_test, prob))
```

8. Code Explanation

`iterations` is tree count, `depth` controls symmetric-tree complexity, and `verbose=False` quiets logs. With a pandas DataFrame, pass `cat_features` and avoid manually target-encoding categories.

9. Training / Evaluation

Use an evaluation set and `early_stopping_rounds`. Tune iterations/rate/depth and `l2_leaf_reg`; validate categories unseen in training and use stratified/time splits appropriate to data generation.

10. Complexity and Cost

Efficient CPU/GPU training; categorical processing adds cost but may replace wide one-hot matrices. Inference is an ensemble traversal.

11. Common Use Cases

Retail/customer data, fraud, ads, pricing, and any table with many string/category columns.

12. Common Mistakes

Manually target-encoding with leakage, converting categories to arbitrary integers without declaring them categorical, using test data in category statistics, and ignoring unknown category behavior.

13. Edge Cases / Limitations

For purely numeric simple data it may offer little benefit over alternatives. Very high-cardinality identifiers can still encode leakage or memorization; validate by entity/time.

14. Variations

- **Ordered boosting:** default leakage-resistant concept; placement-important.
- **Plain boosting:** faster on some large/GPU workloads.
- **CatBoost ranking:** use for ranking/recommendation tasks.

15. Related Topics

CatBoost competes with XGBoost/LightGBM. Its key distinction is ordered treatment of categorical features, whereas naive target encoding must be out-of-fold.

16. Interview Questions

Question	Answer
Why CatBoost?	Strong boosting with native categorical handling and reduced target leakage.
What is target leakage in encoding?	A row's label influences its own feature representation.

Question	Answer
How does ordered encoding help?	It uses only preceding rows in a permutation.
What are symmetric trees?	Each depth uses one shared split condition across nodes.
Need one-hot encoding?	Usually no; provide categorical features natively.
Can category IDs be numeric?	Yes, but explicitly mark them categorical.
Main tuning knobs?	Iterations, learning rate, depth, L2, early stopping.
CatBoost vs LightGBM?	CatBoost is often easier/safer for categories; LightGBM may be faster on large numeric data.
Can identifiers leak?	Yes; entity/time validation is essential.
Why calibration check?	Boosting probabilities need not reflect real event frequencies.

17. Practice Tasks

Compare CatBoost to one-hot LightGBM; intentionally demonstrate leaky target encoding; evaluate unseen categories; tune depth; use SHAP for explanations.

18. Project Ideas

- **Retail churn:** CatBoost on demographics/products, Telco/retail dataset; highlights categories.
- **Used-car price class:** categorical make/model plus numeric features, CarDekho; realistic feature mix.
- **Fraud propensity:** merchant/category data, IEEE-CIS subset; leakage-aware validation.

19. Quick Revision

Key idea: categorical-aware ordered boosting. Formula: prior-only category mean. Use: category-heavy tables. Metrics: AUC/F1. Trap: IDs and split leakage. One-liner: "GBDT that learns categories without naïve target leakage."

20. Final Cheat Sheet

Definition: ordered categorical gradient boosting. Input/output: mixed table → probability/class. Steps: ordered stats, symmetric trees, boost. Hyperparameters: iterations, rate, depth, L2. Pros: categories; cons: tuning/cost. Best: heterogeneous tabular data.

K-Means Clustering

1. Overview

K-Means is a centroid-based clustering algorithm that partitions unlabeled data into **K** groups. It is useful when you expect compact, roughly spherical clusters and want a simple, fast baseline.

Real-world uses include customer segmentation, image color quantization, document grouping after embeddings, feature engineering, and data exploration.

2. Intuition

Imagine placing K magnets on a table of data points. Each point attaches to the nearest magnet. Then each magnet moves to the average position of its attached points. Repeat until the magnets stop moving.

Example: customers can be clustered by annual income and spending score into low, medium, and high-value groups.

3. Prerequisites

- Euclidean distance
- Mean and variance
- Feature scaling
- Basic NumPy and scikit-learn
- Understanding of local minima

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Centroid	Mean vector of a cluster	Defines cluster center	Average customer profile	Why mean makes K-Means sensitive to outliers
Assignment step	Assign each point to nearest centroid	Creates clusters	Point joins closest group	Distance metric assumptions
Update step	Recompute centroids	Improves fit	New cluster average	Why objective decreases
Inertia/WCSS	Sum of squared distances to centroids	Main objective	Lower inertia means tighter clusters	Elbow method
Initialization	Starting centroid choice	Affects final clusters	Random vs k-means++	Local optimum problem

5. Algorithm / Working Process

Input: feature matrix X and number of clusters K .

Steps:

1. Initialize K centroids.
2. Assign every point to the nearest centroid.
3. Recompute each centroid as the mean of assigned points.
4. Repeat assignment and update until convergence.

5. Output cluster labels and centroid locations.

Training is unsupervised because labels are not used. Inference assigns a new point to its nearest learned centroid.

6. Mathematical Foundation

K-Means minimizes within-cluster sum of squares:

$$J = \sum_{i=1}^n \sum_{k=1}^K z_{ik} ||x_i - \mu_k||^2$$

where $z_{ik} = 1$ if point x_i belongs to cluster k , otherwise 0 , and μ_k is centroid k .

Centroid update:

$$\mu_k = (1 / |C_k|) \sum_{x_i \in C_k} x_i$$

The algorithm monotonically decreases the objective but can converge to a local minimum.

7. Practical Implementation

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.2, random_state=42)
X = StandardScaler().fit_transform(X)

model = KMeans(n_clusters=4, init="k-means++", n_init=10, random_state=42)
labels = model.fit_predict(X)

print("Centroids:\n", model.cluster_centers_)
print("Inertia:", model.inertia_)
print("Silhouette:", silhouette_score(X, labels))
```

8. Code Explanation

`make_blobs` creates synthetic clustered data. `StandardScaler` prevents features with large numeric ranges from dominating distance. `KMeans` learns centroids and labels. `inertia_` measures compactness, while `silhouette` checks whether clusters are separated.

9. Training / Evaluation

There is no train/test split requirement for pure clustering, but you can split if using clusters as downstream features. Common metrics include inertia, silhouette score, Davies-Bouldin index, Calinski-Harabasz score, and domain validation.

Important hyperparameters: `n_clusters`, `init`, `n_init`, `max_iter`, `tol`.

Improve performance by scaling features, removing outliers, using PCA for noisy high-dimensional data, and trying different `K` values.

10. Complexity and Cost

Training complexity is approximately:

$$O(n * K * d * i)$$

where `n` is samples, `d` is dimensions, and `i` is iterations. Inference is $O(Kd)$ per point. Memory is $O(n + Kd)$. CPU is usually enough.

11. Common Use Cases

- Customer segmentation
- Image compression
- Document clustering over embeddings
- Market basket user grouping
- Geospatial region grouping
- Prototype-based feature engineering

12. Common Mistakes

- Forgetting feature scaling
- Choosing `K` only by inertia
- Using K-Means on non-spherical clusters
- Ignoring outliers
- Treating cluster IDs as ordered labels
- Assuming clusters are stable across random seeds
- Using K-Means on categorical data without proper encoding

13. Edge Cases / Limitations

K-Means performs poorly with non-convex clusters, unequal density, heavy outliers, many categorical variables, and high-dimensional sparse data. It also requires `K` in advance.

14. Variations

Variation	What Changes	When To Use	Placement Importance
MiniBatch K-Means	Updates centroids using batches	Large datasets	High

Variation	What Changes	When To Use	Placement Importance
K-Medoids	Uses real points as centers	Outlier robustness	Medium
Kernel K-Means	Clusters in implicit feature space	Nonlinear clusters	Medium
Spherical K-Means	Uses cosine similarity	Text embeddings	High

15. Related Topics

K-Means relates to GMMs because both use cluster centers, but GMMs give soft probabilistic assignments. It relates to PCA because PCA can reduce dimensions before clustering. It differs from DBSCAN because K-Means assumes compact clusters and requires K .

16. Interview Questions

1. What does K-Means optimize? It minimizes within-cluster squared distance to centroids.
2. Why scale features? Distance-based algorithms are dominated by large-scale features.
3. Why can K-Means converge to different answers? Random initialization can lead to different local minima.
4. What is k-means++? A smarter initialization that spreads initial centroids.
5. How do you choose K ? Elbow, silhouette, gap statistic, and domain usefulness.
6. Why is K-Means sensitive to outliers? Centroids are means, and means shift under extreme values.
7. Can K-Means handle categorical data? Not directly; use proper encodings or k-modes.
8. What happens if a cluster becomes empty? Implementations reinitialize or handle the empty centroid.
9. Is K-Means supervised? No, it uses no target labels.
10. K-Means vs GMM? K-Means gives hard spherical clusters; GMM gives soft elliptical probabilistic clusters.

17. Practice Tasks

- Implement K-Means from scratch using NumPy.
- Cluster Mall Customers dataset and explain segments.
- Compare elbow and silhouette for $K=2..10$.
- Debug a result where one feature dominates because scaling was skipped.
- Extend with PCA visualization.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Customer Segmentation	Groups users by spending behavior	pandas, sklearn	Mall Customers	Business ML
Image Color Compressor	Reduces image colors to K centroids	OpenCV, sklearn	Any images	Visual ML
Embedding Cluster Explorer	Clusters sentence embeddings	sentence-transformers, sklearn	News/articles	NLP + unsupervised

19. Quick Revision

- Key idea: assign points to nearest centroid, update centroids.
- Main formula: minimize $\sum ||x_i - \mu_k||^2$.
- When to use: compact numeric clusters.
- Metrics: inertia, silhouette.
- Common traps: no scaling, wrong K , outliers.
- Interview one-liner: K-Means is fast centroid-based clustering for spherical groups.

20. Final Cheat Sheet

Item	Summary
Definition	Partitions data into K centroid-based clusters
Input/output	X -> cluster labels and centroids
Main steps	initialize, assign, update, repeat
Hyperparameters	$n_clusters$, $init$, n_init , max_iter
Metrics	inertia, silhouette
Pros	simple, fast, scalable
Cons	needs K , sensitive to scale/outliers
Best use	quick segmentation baseline

K-Medoids

1. Overview

K-Medoids partitions data into K clusters like K-Means, but each center is an actual observation (a **medoid**) minimizing within-cluster dissimilarity. Because arbitrary pairwise distances can be used and centers cannot be pulled to impossible averages, it is more robust to outliers and mixed/non-Euclidean data.

Applications include robust customer segmentation, facility-location prototypes, biological sequence grouping, and clustering from a precomputed distance matrix.

2. Intuition

If a group must choose a meeting representative, K-Means invents an "average person"; K-Medoids chooses the real member whose total distance to everyone else is smallest. An extreme member can move the average substantially but rarely becomes the representative.

3. Prerequisites

- Clustering, distances/dissimilarities, and feature scaling
- K-Means and the distinction between mean and median robustness
- Pairwise distance matrices and basic combinatorial optimization

- Silhouette score and unsupervised model selection

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Medoid	Real point with smallest cluster dissimilarity	Interpretable, robust prototype	An actual customer	Medoid vs centroid
Dissimilarity	Any meaningful pairwise cost	Enables Manhattan/Gower/edit distance	Sequence edit distance	Need not be Euclidean
PAM	Partitioning Around Medoids swap algorithm	Standard exact-ish approach	Swap medoid with non-medoid	Computational bottleneck
Assignment	Join nearest medoid	Forms clusters	Nearest representative	Same hard-label idea as K-Means
Swap cost	Objective change after replacing a medoid	Finds better prototypes	Replace customer A with B	Local optimum

5. Algorithm / Working Process

Input: observations or an $n \times n$ dissimilarity matrix and K .

1. Select K initial medoids (randomly or with a smarter initializer).
2. Assign each observation to its nearest medoid.
3. For every medoid/non-medoid pair, evaluate the objective after swapping them.
4. Apply the swap with the greatest reduction.
5. Repeat until no swap improves cost or an iteration limit is reached.
6. Output medoid indices, hard labels, and total dissimilarity.

Inference assigns a new point to the nearest learned medoid, provided distances from the new point to medoids can be computed.

6. Mathematical Foundation

For medoid set M , $|M|=K$, minimize

$$J(M) = \sum_{i=1}^n \min_{m \in M} d(x_i, x_m).$$

The medoid of a fixed cluster C_k is

$$m_k = \arg \min_{x_j \in C_k} \sum_{x_i \in C_k} d(x_i, x_j).$$

Unlike the K-Means squared-Euclidean objective, no arithmetic mean is required. Robustness comes from choosing a data point and often using unsquared distances; it is not absolute immunity to many or adversarial outliers.

7. Practical Implementation

```
# pip install scikit-learn-extra
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn_extra.cluster import KMedoids

X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.3, random_state=42)
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KMedoids(
    n_clusters=4, metric="manhattan", method="pam",
    init="k-medoids++", random_state=42
)
train_labels = model.fit_predict(X_train)
test_labels = model.predict(X_test)

print("Medoid rows:", model.medoid_indices_)
print("Objective:", model.inertia_)
print("Train silhouette:", silhouette_score(X_train, train_labels,
metric="manhattan"))
print("Assigned test clusters:", test_labels[:10])
```

8. Code Explanation

The scaler is fitted only on training data. Manhattan distance is used consistently in clustering and silhouette evaluation. `method="pam"` evaluates medoid swaps; `k-medoids++` improves initialization. `medoid_indices_` identifies real training examples, which can be shown as representative cases.

9. Training / Evaluation

There are no target labels. Select `K`, distance, and initialization using silhouette, stability under bootstrap/resampling, objective curves, and domain usefulness. If ground-truth classes exist only for evaluation, ARI/NMI may be reported but should not drive an allegedly unsupervised pipeline without disclosure. Validate preprocessing and distance choice; scaling numeric features is usually essential.

10. Complexity and Cost

Classic PAM is roughly $O(K(n-K)^2)$ per iteration and storing all pairwise distances costs $O(n^2)$ memory. Prediction is $O(Kd)$ when distances are computed from features. This is much more expensive than K-Means, so CPU use is practical mainly for modest n ; CLARA/CLARANS approximate large-data solutions.

11. Common Use Cases

- Robust segmentation with noisy numeric features
- Clustering using Manhattan, Gower, edit, or precomputed distances
- Selecting real representative examples for human review
- Facility-location and prototype selection
- Biomedical or sequence clustering

12. Common Mistakes

- Calling a medoid a mean or assuming it may lie outside the dataset
- Forgetting scaling before Manhattan/Euclidean distance
- Using Euclidean distance on mixed categorical data without justification
- Applying PAM to millions of rows without approximation
- Choosing K solely from decreasing inertia
- Evaluating silhouette with a different metric from training

13. Edge Cases / Limitations

It still requires K , produces hard partitions, and tends to prefer compact clusters under the chosen metric. PAM can converge to local optima and is expensive. Duplicate points may produce equally good medoids; arbitrary medoid identity should not be overinterpreted. A bad distance makes even a correctly optimized clustering meaningless.

14. Variations

Variation	Change	When to use	Importance
PAM	Exhaustive improving swaps	Small/medium datasets	Placement: high
CLARA	PAM on repeated samples	Larger n	Placement/project: high
CLARANS	Randomized neighbor search	Larger combinatorial search	Medium
FastPAM	Reuses swap computations	Faster exact PAM-style fitting	Project: medium
K-Medians	Coordinate-wise median, not actual point	Manhattan numeric data	Interview distinction: high

15. Related Topics

K-Means uses centroids and squared Euclidean distance; K-Medoids uses observed representatives and arbitrary dissimilarity. K-Medians uses component-wise medians that need not be observed rows. DBSCAN

avoids specifying **K** and can discover noise/non-convex clusters. Hierarchical clustering also accepts precomputed distances but returns a dendrogram rather than one mandatory partition.

16. Interview Questions

1. **What is a medoid?** The cluster observation minimizing total dissimilarity to other members.
2. **K-Means vs K-Medoids?** Mean centroid/squared Euclidean and speed versus real prototype/arbitrary metric and robustness.
3. **Why more robust to outliers?** Extreme values cannot pull a medoid continuously as they pull a mean.
4. **What does PAM do?** Iteratively swaps medoids and non-medoids when the objective improves.
5. **Can it use categorical data?** Yes, with a defensible dissimilarity such as Gower or matching distance.
6. **Does it find non-convex shapes?** Generally not; nearest-medoid Voronoi partitions remain compact under the metric.
7. **How choose K?** Silhouette/stability plus domain interpretability.
8. **Why is it slower than K-Means?** Candidate medoid swaps and pairwise distances are expensive.
9. **K-Medoids vs K-Medians?** The former center is a real row; the latter is a coordinate-wise median.
10. **Can new points be predicted?** Yes, by nearest medoid if their distances are available.
11. **What if only a distance matrix exists?** Fit with `metric="precomputed"`; out-of-sample assignment needs new-to-medoid distances.
12. **Main scaling alternative?** CLARA samples the dataset and tests sample-derived medoids broadly.

17. Practice Tasks

- Implement PAM swap search on a tiny distance matrix.
- Inject extreme outliers and compare centroid/medoid movement.
- Cluster mixed data with Gower distance and explain feature weighting.
- Benchmark PAM versus CLARA-style sampling.
- Debug inconsistent metric use between training and silhouette scoring.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Representative Customer Finder	Segments users and surfaces actual representative profiles	pandas, sklearn-extra	Mall/marketing data	Interpretability
Hospital Similarity Explorer	Clusters hospitals from mixed features using Gower distance	pandas, gower, sklearn-extra	CMS public data	Mixed-data modeling
Robust Location Planner	Chooses representative service locations under travel distance	GeoPandas, sklearn-extra	OpenStreetMap-derived points	Optimization + geospatial ML

19. Quick Revision

- **Key idea:** choose real observations minimizing within-cluster dissimilarity.
- **Main formula:** $\sum_i \min_{(m \in M)} d(x_i, x_m)$.

- **When to use:** outliers, arbitrary distances, interpretable prototypes.
- **Metrics:** objective, silhouette, stability, domain utility.
- **Traps:** quadratic distance cost, wrong metric, unscaled inputs.
- **Interview one-liner:** "K-Medoids is a robust prototype-based partitioner whose centers are actual data points."

20. Final Cheat Sheet

Item	Summary
Definition	Partitioning around real representative points
Input/output	Features/distance matrix -> labels and medoids
Main steps	Initialize, assign, evaluate swaps, improve, stop
Hyperparameters	K , metric, initializer, PAM/approximation
Metrics	Dissimilarity objective, silhouette, stability
Pros	Robust, interpretable, arbitrary distances
Cons	Expensive, requires K , local optimum
Best use cases	Modest datasets needing robust real prototypes

Hierarchical Clustering

1. Overview

Hierarchical clustering builds a tree of clusters instead of forcing one fixed number of clusters immediately. The tree is called a dendrogram. It is useful when you want to inspect cluster structure at multiple granularities.

It is used in biology, document grouping, customer segmentation, taxonomy discovery, and exploratory analysis.

2. Intuition

Think of arranging people into friend groups. At first everyone is alone. Then the two most similar people merge. Next, similar pairs or groups merge. Eventually everyone belongs to one big group. Cutting the tree at a chosen height gives clusters.

3. Prerequisites

- Distance metrics
- Linkage criteria
- Matrix operations
- Basic clustering metrics
- Dendrogram interpretation

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Agglomerative	Bottom-up merging	Most common approach	Start with each point alone	Complexity
Divisive	Top-down splitting	Less common	Start with all points	Harder optimization
Linkage	Distance between clusters	Controls cluster shape	Single vs complete	Chaining problem
Dendrogram	Tree visualization	Helps choose cluster count	Cut at height 5	Interpretability
Distance metric	Similarity definition	Drives all merges	Euclidean/cosine	Metric choice

5. Algorithm / Working Process

Input: data matrix X , distance metric, linkage method.

Steps:

1. Treat each point as its own cluster.
2. Compute pairwise distances between clusters.
3. Merge the two closest clusters.
4. Update distances using linkage.
5. Repeat until one cluster remains or desired cluster count is reached.
6. Cut the dendrogram to obtain final labels.

6. Mathematical Foundation

Common linkage formulas:

Single linkage: $d(A, B) = \min d(a, b)$
Complete linkage: $d(A, B) = \max d(a, b)$
Average linkage: $d(A, B) = \text{mean } d(a, b)$
Ward linkage: merge that minimizes increase in within-cluster variance

Ward's method approximately minimizes:

$$\Delta(A, B) = (|A| \cdot |B| / (|A| + |B|)) \cdot ||\mu_A - \mu_B||^2$$

7. Practical Implementation

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=300, centers=3, random_state=7)
X = StandardScaler().fit_transform(X)

model = AgglomerativeClustering(n_clusters=3, linkage="ward")
labels = model.fit_predict(X)

print("Silhouette:", silhouette_score(X, labels))
print("First labels:", labels[:10])
```

8. Code Explanation

The code creates numeric cluster data, scales it, then applies agglomerative clustering. `linkage="ward"` is a strong default for Euclidean numeric data because it favors compact clusters.

9. Training / Evaluation

Evaluation is similar to other clustering methods: silhouette score, dendrogram inspection, cluster stability, and domain review. There is no gradient training. Hyperparameters include `n_clusters`, `distance_threshold`, `metric`, and `linkage`.

10. Complexity and Cost

Hierarchical clustering usually needs pairwise distances, so memory is often $O(n^2)$. Time complexity is commonly $O(n^2 \log n)$ or worse depending on implementation. It is not ideal for very large datasets.

11. Common Use Cases

- Gene expression clustering
- Topic hierarchy discovery
- Customer segmentation with dendrogram explainability
- Document clustering
- Taxonomy generation

12. Common Mistakes

- Using it on huge datasets without sampling
- Choosing linkage blindly
- Misreading dendrogram height
- Forgetting scaling
- Using Ward linkage with non-Euclidean metrics
- Assuming dendrogram cut is objectively correct

13. Edge Cases / Limitations

It struggles with large data, noisy distances, high-dimensional data, and irreversible early merges. Single linkage can create long chained clusters.

14. Variations

Variation	What Changes	When To Use	Importance
Agglomerative	Bottom-up	Standard use	High
Divisive	Top-down	Taxonomy splitting	Medium
BIRCH	Clustering feature tree	Large data	Medium
Ward clustering	Variance-minimizing merges	Compact numeric clusters	High

15. Related Topics

Compared with K-Means, hierarchical clustering does not require choosing **K** before building the tree. Compared with DBSCAN, it does not explicitly model noise. Compared with spectral clustering, it is simpler but less powerful for graph-like structures.

16. Interview Questions

1. What is a dendrogram? A tree showing cluster merges and merge distances.
2. Agglomerative vs divisive? Agglomerative merges bottom-up; divisive splits top-down.
3. What is linkage? A rule for distance between clusters.
4. Why is single linkage risky? It can chain points through bridges.
5. What does Ward linkage optimize? It minimizes increase in within-cluster variance.
6. Does it need **K**? Not before building; final clusters come from a cut.
7. Why is it expensive? Pairwise distances need large time and memory.
8. How choose clusters? Cut dendrogram, use distance threshold, or metrics.
9. Is scaling needed? Yes for distance-based numeric clustering.
10. When prefer hierarchical over K-Means? When cluster hierarchy and interpretability matter.

17. Practice Tasks

- Plot a dendrogram with SciPy.
- Compare single, complete, average, and Ward linkage.
- Cluster document embeddings using cosine distance.
- Debug chained clusters under single linkage.
- Use a distance threshold instead of fixed **n_clusters**.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Research Paper Taxonomy	Groups papers into topic tree	sklearn, scipy	arXiv metadata	Research tooling

Project	What It Does	Tech Stack	Dataset	Resume Value
Product Category Discovery	Builds category hierarchy	pandas, sklearn	E-commerce catalog	Business ML
Gene Cluster Explorer	Clusters gene expression profiles	scipy, seaborn	UCI gene data	Bioinformatics

19. Quick Revision

- Key idea: build cluster tree using repeated merges.
- Main formula: linkage distance.
- When to use: small/medium data needing hierarchy.
- Metrics: silhouette, dendrogram height.
- Common traps: scaling, linkage mismatch, high cost.
- Interview one-liner: hierarchical clustering gives a multi-resolution cluster tree.

20. Final Cheat Sheet

Item	Summary
Definition	Tree-based clustering
Input/output	X -> dendrogram/labels
Main steps	compute distances, merge closest clusters
Hyperparameters	linkage, metric, n_clusters , threshold
Metrics	silhouette, dendrogram inspection
Pros	interpretable hierarchy
Cons	expensive, early mistakes irreversible
Best use	taxonomy and exploratory clustering

DBSCAN

1. Overview

DBSCAN means Density-Based Spatial Clustering of Applications with Noise. It finds dense regions separated by sparse regions and marks isolated points as noise. It does not require pre-selecting the number of clusters.

It is useful for geospatial clustering, anomaly removal, sensor data, fraud pre-filtering, and non-spherical cluster discovery.

2. Intuition

Imagine dots on a map. A cluster is an area where each dot has enough nearby neighbors. Sparse dots outside dense areas are treated as noise.

3. Prerequisites

- Distance metrics
- Nearest neighbors
- Density intuition
- Feature scaling
- Clustering metrics

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
<code>eps</code>	Neighborhood radius	Defines local density	Points within 0.5 units	Hardest hyperparameter
<code>min_samples</code>	Minimum neighbors for core point	Controls density threshold	At least 5 neighbors	Noise sensitivity
Core point	Dense point	Expands clusters	In crowded area	Cluster formation
Border point	Near core but not dense itself	Belongs to cluster	Edge of group	Assignment ambiguity
Noise point	Not density-reachable	Outlier	Isolated transaction	Anomaly detection

5. Algorithm / Working Process

Input: `X`, `eps`, `min_samples`.

Steps:

1. For each point, find neighbors within `eps`.
2. Mark point as core if neighbors $\geq$ `min_samples`.
3. Start from an unvisited core point.
4. Expand cluster through density-reachable core points.
5. Assign border points to clusters.
6. Mark unreachable points as noise label `-1`.

6. Mathematical Foundation

Epsilon neighborhood:

$$N_{\text{eps}}(x) = \{y \text{ in } X : \text{distance}(x, y) \leq \text{eps}\}$$

Core condition:

$$|N_{\text{eps}}(x)| \geq \text{min_samples}$$

Density reachability means q is reachable from p through a chain of core points.

7. Practical Implementation

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

X, _ = make_moons(n_samples=500, noise=0.07, random_state=42)
X = StandardScaler().fit_transform(X)

model = DBSCAN(eps=0.25, min_samples=5)
labels = model.fit_predict(X)

non_noise = labels != -1
print("Clusters:", len(set(labels)) - (1 if -1 in labels else 0))
print("Noise points:", (~non_noise).sum())
print("Silhouette:", silhouette_score(X[non_noise], labels[non_noise]))
```

8. Code Explanation

`make_moons` creates non-spherical clusters where K-Means often fails. DBSCAN discovers curved dense regions. Noise points get label `-1`, so they are excluded from silhouette scoring.

9. Training / Evaluation

DBSCAN does not train parameters. It computes neighborhoods and cluster expansion. Use k-distance plots to tune `eps`, and domain knowledge for `min_samples`. Evaluation uses noise ratio, cluster count, silhouette on non-noise points, and manual inspection.

10. Complexity and Cost

With indexing, DBSCAN is often near $O(n \log n)$ for low-dimensional data. Without efficient neighbor search, it can be $O(n^2)$. High-dimensional distances reduce effectiveness.

11. Common Use Cases

- GPS location clustering
- Fraud/anomaly pre-filtering
- Duplicate entity grouping
- Sensor event clustering
- Shape-based clustering

12. Common Mistakes

- Not scaling features
- Setting `eps` too large and merging everything
- Setting `eps` too small and labeling everything noise

- Using DBSCAN in very high dimensions without embeddings/reduction
- Comparing labels directly across runs or datasets
- Ignoring variable-density clusters

13. Edge Cases / Limitations

DBSCAN struggles when clusters have different densities, distances become meaningless in high dimensions, or data has no clear density gap.

14. Variations

Variation	What Changes	When To Use	Importance
HDBSCAN	Hierarchical density clustering	Variable density	High
OPTICS	Orders points by density reachability	Unknown eps	Medium
ST-DBSCAN	Adds spatial-temporal constraints	GPS/time data	Medium

15. Related Topics

DBSCAN contrasts with K-Means because it finds arbitrary shapes and noise. It connects to anomaly detection because noise labels can be treated as outliers. HDBSCAN is a stronger practical extension.

16. Interview Questions

1. What does DBSCAN stand for? Density-Based Spatial Clustering of Applications with Noise.
2. Does DBSCAN need **K**? No.
3. What is a core point? A point with at least **min_samples** neighbors inside **eps**.
4. What is a border point? A non-core point reachable from a core point.
5. What label marks noise in sklearn? **-1**.
6. How choose **eps**? Use k-distance plot and domain knowledge.
7. Why scale data? Neighborhood distances depend on feature scales.
8. DBSCAN vs K-Means? DBSCAN handles noise and arbitrary shapes; K-Means is faster for spherical clusters.
9. Main weakness? Variable density and high dimensions.
10. Can DBSCAN predict new points naturally? Standard DBSCAN does not learn a centroid model; use approximate assignment if needed.

17. Practice Tasks

- Cluster two-moons data.
- Draw a k-distance plot for **eps**.
- Compare DBSCAN and K-Means on non-spherical data.
- Analyze noise points as anomalies.
- Try HDBSCAN on variable-density data.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Hotspot Finder	Finds dense event locations	sklearn, geopandas	NYC taxi	Geospatial ML
Fraud Burst Detector	Finds dense suspicious events and outliers	pandas, sklearn	Credit card fraud	Risk analytics
Sensor Event Grouper	Groups machine events	sklearn	NASA bearing/sensor data	Industrial AI

19. Quick Revision

- Key idea: dense regions are clusters; sparse points are noise.
- Main formula: $|N_{\text{eps}}(x)| \geq \text{min_samples}$.
- When to use: arbitrary shapes with outliers.
- Metrics: noise ratio, silhouette, visual checks.
- Common traps: bad eps , no scaling.
- Interview one-liner: DBSCAN clusters by density and identifies noise automatically.

20. Final Cheat Sheet

Item	Summary
Definition	Density-based clustering
Input/output	$X \rightarrow$ labels with -1 noise
Main steps	find neighborhoods, expand core points
Hyperparameters	eps , min_samples , metric
Metrics	cluster count, noise rate, silhouette
Pros	arbitrary shapes, no K , detects noise
Cons	sensitive to density settings
Best use	spatial/noisy clustering

Gaussian Mixture Models

1. Overview

Gaussian Mixture Models, or GMMs, are probabilistic clustering models that represent data as a mixture of multiple Gaussian distributions. Unlike K-Means, GMMs provide soft cluster membership probabilities.

2. Intuition

Instead of saying each point belongs fully to one cluster, GMM says a point may be 70% likely from one Gaussian and 30% from another. This is useful when clusters overlap.

3. Prerequisites

- Gaussian distribution
- Probability density
- Maximum likelihood
- Expectation-Maximization
- Covariance matrices

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Mixture component	One Gaussian in the model	Represents a cluster	Segment A	Soft clustering
Mixing weight	Prior probability of component	Cluster size	40% component 1	Must sum to 1
Mean	Component center	Location	Average profile	Like centroid
Covariance	Component shape/spread	Elliptical clusters	Correlated features	More flexible than K-Means
Responsibility	Probability point belongs to component	Soft label	<code>gamma_ik=0.8</code>	E-step

5. Algorithm / Working Process

Input: X , number of components K .

Steps:

1. Initialize means, covariances, and weights.
2. E-step: compute responsibilities for each point-component pair.
3. M-step: update parameters using responsibilities.
4. Repeat until log-likelihood converges.
5. Output probabilities, labels, and density estimates.

Inference computes component probabilities for new points.

6. Mathematical Foundation

Mixture density:

$$p(x) = \sum_{k=1}^K \pi_k N(x \mid \mu_k, \Sigma_k)$$

Responsibility:

$$\gamma_{ik} = \pi_k N(x_i | \mu_k, \Sigma_k) / \sum_j \pi_j N(x_i | \mu_j, \Sigma_j)$$

Mean update:

$$\mu_k = \sum_i \gamma_{ik} x_i / \sum_i \gamma_{ik}$$

Log-likelihood:

$$L = \sum_i \log(\sum_k \pi_k N(x_i | \mu_k, \Sigma_k))$$

7. Practical Implementation

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture

X, _ = make_blobs(n_samples=500, centers=3, cluster_std=1.5, random_state=42)
X = StandardScaler().fit_transform(X)

gmm = GaussianMixture(n_components=3, covariance_type="full", random_state=42)
gmm.fit(X)

labels = gmm.predict(X)
probs = gmm.predict_proba(X)
log_density = gmm.score_samples(X)

print("Means:\n", gmm.means_)
print("First probabilities:\n", probs[:3])
print("BIC:", gmm.bic(X))
```

8. Code Explanation

`GaussianMixture` fits Gaussian components using EM. `predict` gives the most likely component. `predict_proba` gives soft assignments. `score_samples` returns log-density, useful for anomaly detection.

9. Training / Evaluation

Evaluate with log-likelihood, BIC, AIC, silhouette on hard labels, and domain validation. Select `n_components` using BIC/AIC and business interpretability.

10. Complexity and Cost

Full covariance GMMs can be expensive in high dimensions because covariance matrices are $d \times d$. Each EM iteration costs roughly $O(n K d^2)$ for full covariance. Diagonal covariance reduces cost.

11. Common Use Cases

- Soft customer segmentation
- Density estimation
- Anomaly detection
- Speaker modeling
- Image segmentation
- Probabilistic clustering

12. Common Mistakes

- Forgetting scaling
- Using too many components
- Ignoring covariance type
- Assuming Gaussian clusters when data is not Gaussian
- Not checking convergence
- Confusing probability density with probability mass

13. Edge Cases / Limitations

GMMs struggle with non-Gaussian shapes, high dimensions, singular covariance matrices, and strong outliers. EM can converge to local optima.

14. Variations

Variation	What Changes	When To Use	Importance
Full covariance	Arbitrary ellipses	Flexible clusters	High
Diagonal covariance	Independent features per component	High-dimensional data	High
Tied covariance	Shared covariance	Small data	Medium
Bayesian GMM	Prior over components	Unknown cluster count	Medium

15. Related Topics

GMM generalizes K-Means: K-Means is like a limiting case with equal spherical covariance and hard assignments. GMMs also connect to anomaly detection through low-density scoring.

16. Interview Questions

1. What is a GMM? A mixture of Gaussian distributions.
2. What is soft clustering? Assigning probabilities instead of hard labels.
3. What algorithm trains GMMs? Expectation-Maximization.
4. What happens in E-step? Compute responsibilities.
5. What happens in M-step? Update means, covariances, and weights.

6. GMM vs K-Means? GMM is probabilistic and handles elliptical overlap.
7. How choose components? BIC, AIC, validation, domain sense.
8. What is covariance type? Constraint on component covariance matrices.
9. Can GMM detect anomalies? Yes, low likelihood points are suspicious.
10. Main weakness? Gaussian assumption and local optima.

17. Practice Tasks

- Fit GMM to blobs and plot probability contours.
- Compare covariance types.
- Use BIC to select K .
- Detect anomalies using low log-density.
- Compare GMM and K-Means on overlapping clusters.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Soft Segmenter	Gives customer membership probabilities	sklearn, pandas	Mall customers	Business interpretability
Density Anomaly Detector	Flags low-density records	sklearn	Credit card fraud	Risk modeling
Image Color Segmenter	Probabilistic pixel segmentation	OpenCV, sklearn	Images	CV fundamentals

19. Quick Revision

- Key idea: data comes from a mixture of Gaussians.
- Main formula: $p(\mathbf{x}) = \sum p_i \cdot N(\mathbf{x} | \mu_i, \Sigma_i)$.
- When to use: overlapping elliptical clusters.
- Metrics: log-likelihood, BIC, AIC.
- Common traps: too many components, bad covariance choice.
- Interview one-liner: GMM is soft probabilistic clustering trained by EM.

20. Final Cheat Sheet

Item	Summary
Definition	Probabilistic mixture of Gaussians
Input/output	$\mathbf{X}$ -> component probabilities
Main steps	E-step, M-step, repeat
Hyperparameters	components, covariance type
Metrics	log-likelihood, BIC, AIC
Pros	soft labels, elliptical clusters

Item	Summary
Cons	Gaussian assumption, expensive covariance
Best use	soft clustering and density estimation

PCA

1. Overview

Principal Component Analysis reduces dimensionality by projecting data onto directions of maximum variance. It is one of the most important unsupervised techniques for compression, denoising, visualization, and preprocessing.

2. Intuition

If a cloud of points forms a stretched ellipse, PCA finds the longest direction first. That direction captures the most variation. The second direction is perpendicular and captures the next most variation.

3. Prerequisites

- Linear algebra: vectors, matrices, eigenvectors
- Variance and covariance
- Standardization
- Matrix decomposition
- Basic visualization

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Principal component	Direction of maximum variance	New feature axis	PC1 of height/weight	Eigenvectors
Explained variance	Variance captured by each PC	Choose dimensions	95% variance retained	Scree plot
Orthogonality	PCs are perpendicular	Removes redundancy	PC1 independent direction from PC2	Decorrelation
Projection	Mapping data to PCs	Dimensionality reduction	100D to 2D	Information loss
Reconstruction	Mapping back approximately	Compression quality	Image compression	Error tradeoff

5. Algorithm / Working Process

Input: numeric matrix X .

Steps:

1. Center features by subtracting mean.
2. Usually scale features to unit variance.
3. Compute covariance matrix or use SVD.
4. Find eigenvectors/eigenvalues.
5. Sort components by eigenvalue.
6. Project data onto top k components.

Output: lower-dimensional representation and components.

6. Mathematical Foundation

Covariance matrix:

$$\text{Sigma} = (1 / (n - 1)) X_{\text{centered}}^T X_{\text{centered}}$$

Eigen decomposition:

$$\text{Sigma } v_j = \lambda_j v_j$$

Projection:

$$Z = X_{\text{centered}} W_k$$

Explained variance ratio:

$$\lambda_j / \sum_i \lambda_i$$

PCA minimizes reconstruction error among all linear projections of dimension k .

7. Practical Implementation

```
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score

X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.2, random_state=42, stratify=y
)

pipe = make_pipeline(
    StandardScaler(),
    PCA(n_components=0.95, random_state=42),
    LogisticRegression(max_iter=2000)
)

pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("PCA components:", pipe.named_steps["pca"].n_components_)
```

8. Code Explanation

The pipeline prevents leakage by fitting scaling and PCA only on training data. `n_components=0.95` keeps enough components to explain 95% of variance. Logistic regression then trains on compressed features.

9. Training / Evaluation

For preprocessing, fit PCA on training data only. Evaluate downstream task performance, explained variance, reconstruction error, or visualization usefulness. Important hyperparameters: `n_components`, whitening, solver.

10. Complexity and Cost

Full PCA via SVD can cost about $O(\min(n \cdot d^2, d \cdot n^2))$. Memory depends on `X` and components. Randomized PCA is better for large data. CPU is enough for common tabular datasets.

11. Common Use Cases

- Dimensionality reduction
- Noise reduction
- 2D/3D visualization
- Feature decorrelation
- Compression
- Preprocessing before clustering

12. Common Mistakes

- Applying PCA before train/test split
- Not scaling features
- Assuming high variance always means high predictive value
- Using PCA for nonlinear manifolds
- Interpreting PCs as original features without checking loadings
- Keeping too few components

13. Edge Cases / Limitations

PCA is linear, sensitive to outliers, and variance-focused rather than label-focused. It can hurt performance if low-variance features are predictive.

14. Variations

Variation	What Changes	When To Use	Importance
Kernel PCA	Nonlinear projection through kernels	Nonlinear data	Medium
Incremental PCA	Batch updates	Large data	Medium
Sparse PCA	Sparse component loadings	Interpretability	Medium
Randomized PCA	Approximate fast SVD	Large matrices	High
Whitening	Scales PCs to unit variance	Certain models	Medium

15. Related Topics

PCA differs from t-SNE and UMAP because PCA is linear and preserves global variance. PCA is often used before K-Means, GMMs, or visualization. Autoencoders can be seen as nonlinear learned dimensionality reduction.

16. Interview Questions

1. What does PCA do? Projects data onto directions of maximum variance.
2. Why center data? PCA directions depend on covariance around the mean.
3. Why scale data? Large-scale features dominate variance.
4. What are principal components? Eigenvectors of covariance matrix.
5. What do eigenvalues represent? Variance explained by components.
6. Is PCA supervised? No.
7. Can PCA improve accuracy? Sometimes, by denoising; sometimes it removes useful signal.
8. PCA vs LDA? PCA is unsupervised variance maximization; LDA is supervised class separation.
9. What is explained variance ratio? Fraction of total variance captured by a component.
10. Why use SVD? It is numerically stable and avoids explicitly forming covariance.

17. Practice Tasks

- Implement PCA with NumPy eigen decomposition.
- Compress digit images and reconstruct them.
- Plot explained variance curve.
- Compare classifier performance before and after PCA.
- Debug data leakage in PCA preprocessing.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Image Compressor	Reconstructs images from top PCs	sklearn, matplotlib	Olivetti faces	Linear algebra + CV

Project	What It Does	Tech Stack	Dataset	Resume Value
PCA Cluster Dashboard	Visualizes clusters in 2D	streamlit, sklearn	Customer data	Explainable ML
Noise Reduction Demo	Removes noise via PCA reconstruction	numpy, sklearn	Digits	Signal processing

19. Quick Revision

- Key idea: keep directions with maximum variance.
- Main formula: $\Sigma v = \lambda v$.
- When to use: compression, denoising, visualization.
- Metrics: explained variance, reconstruction error.
- Common traps: leakage, no scaling, overinterpretation.
- Interview one-liner: PCA is linear dimensionality reduction using eigenvectors of covariance.

20. Final Cheat Sheet

Item	Summary
Definition	Linear dimensionality reduction
Input/output	high-dimensional X -> lower-dimensional Z
Main steps	center, decompose, project
Hyperparameters	<code>n_components</code> , whitening, solver
Metrics	explained variance, reconstruction error
Pros	fast, interpretable, useful preprocessing
Cons	linear, outlier-sensitive
Best use	compression and visualization baseline

t-SNE

1. Overview

t-SNE, or t-distributed Stochastic Neighbor Embedding, is a nonlinear visualization algorithm mainly used to map high-dimensional data into 2D or 3D while preserving local neighborhoods.

It is popular for visualizing embeddings, image features, gene expression data, and hidden neural network representations.

2. Intuition

t-SNE tries to keep close neighbors close. If two points are similar in high-dimensional space, t-SNE strongly prefers them to remain near each other in the 2D map. It cares more about local neighborhoods than exact

global distances.

3. Prerequisites

- Probability distributions
- Pairwise distances
- KL divergence
- Gradient descent intuition
- Embeddings and dimensionality reduction

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Local similarity	Probability of neighbor relation	Preserves neighborhoods	Similar images nearby	Not a clustering algorithm
Perplexity	Effective neighborhood size	Controls local/global balance	5 vs 50 neighbors	Tuning
Student-t distribution	Heavy-tailed low-dimensional similarity	Avoids crowding	Separates clusters visually	Crowding problem
KL divergence	Objective mismatch measure	Optimization target	High-D vs low-D probabilities	Asymmetry
Random seed	Initialization sensitivity	Different layouts possible	Rotated/scattered maps	Reproducibility

5. Algorithm / Working Process

Input: high-dimensional features or embeddings.

Steps:

1. Compute pairwise similarities in high-dimensional space.
2. Convert similarities into probabilities p_{ij} .
3. Initialize low-dimensional points.
4. Compute low-dimensional similarities q_{ij} using Student-t distribution.
5. Minimize KL divergence between P and Q .
6. Output 2D/3D coordinates for visualization.

t-SNE is usually not used for inference on new points.

6. Mathematical Foundation

High-dimensional conditional probability:

$$p_{\{j|i\}} = \exp(-||x_i - x_j||^2 / 2 \sigma_i^2) / \sum_{k \neq i} \exp(-||x_i - x_k||^2 / 2 \sigma_i^2)$$

Low-dimensional similarity:

$$q_{ij} = (1 + ||y_i - y_j||^2)^{-1} / \sum_{k \neq i} (1 + ||y_k - y_i||^2)^{-1}$$

Objective:

$$KL(P || Q) = \sum_i \sum_j p_{ij} \log(p_{ij} / q_{ij})$$

7. Practical Implementation

```
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

X, y = load_digits(return_X_y=True)
X = StandardScaler().fit_transform(X)

# PCA first speeds up t-SNE and removes some noise.
X50 = PCA(n_components=50, random_state=42).fit_transform(X)

tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate="auto",
    init="pca",
    random_state=42
)
coords = tsne.fit_transform(X50)

print(coords[:5])
print(y[:5])
```

8. Code Explanation

Digits are scaled, reduced with PCA, then embedded with t-SNE. PCA is a practical preprocessing step because t-SNE is expensive on high-dimensional noisy data. The output coordinates are for plotting, not classification by themselves.

9. Training / Evaluation

t-SNE does not train a reusable predictive model in standard sklearn usage. Evaluate visually, by neighborhood preservation, and by stability across seeds/perplexities. Avoid using visual clusters as proof of real classes without validation.

10. Complexity and Cost

Exact t-SNE is expensive, around $O(n^2)$. Barnes-Hut or FFT approximations improve scaling. CPU works for thousands of points; large datasets need sampling or faster implementations.

11. Common Use Cases

- Visualizing embeddings
- Inspecting class separability
- Analyzing neural network features
- Bioinformatics visualization
- Debugging representation learning

12. Common Mistakes

- Treating t-SNE as a clustering algorithm
- Interpreting distances between far clusters literally
- Ignoring perplexity sensitivity
- Running directly on raw unscaled features
- Comparing axes as meaningful dimensions
- Overclaiming from pretty plots

13. Edge Cases / Limitations

t-SNE is slow, stochastic, weak at global structure, and awkward for new unseen points. It can create visually separated groups even when the structure is less clear.

14. Variations

Variation	What Changes	When To Use	Importance
Barnes-Hut t-SNE	Approximate nearest interactions	Medium datasets	High
Flt-SNE/openTSNE	Faster approximation	Large visualization	Medium
Parametric t-SNE	Neural net maps points	Need transform for new data	Research

15. Related Topics

t-SNE is related to UMAP, which is often faster and preserves more global structure. PCA is usually used before t-SNE. Unlike clustering methods, t-SNE only provides coordinates for visualization.

16. Interview Questions

1. What is t-SNE used for? High-dimensional visualization.
2. Does t-SNE preserve global distances? Not reliably.

3. What is perplexity? A neighborhood-size parameter.
4. Why use Student-t distribution? Heavy tails reduce crowding.
5. What loss does it minimize? KL divergence between similarity distributions.
6. Should we cluster t-SNE output? Be careful; t-SNE can distort distances.
7. Why run PCA before t-SNE? Speed and denoising.
8. Are axes meaningful? No, orientation and axis values are arbitrary.
9. Can t-SNE transform new points? Standard t-SNE does not naturally support this.
10. t-SNE vs PCA? t-SNE is nonlinear/local; PCA is linear/global variance.

17. Practice Tasks

- Visualize MNIST/digits embeddings.
- Compare perplexity values 5, 30, 50.
- Run t-SNE with and without PCA preprocessing.
- Check stability across random seeds.
- Compare t-SNE and UMAP on the same embeddings.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Embedding Map	Visualizes sentence embeddings	sklearn, sentence-transformers	News titles	NLP interpretability
CNN Feature Explorer	Shows image feature clusters	PyTorch, sklearn	CIFAR-10	Deep learning analysis
Bio Cell Visualizer	Maps gene expression data	scanpy/sklearn	PBMC data	Research flavor

19. Quick Revision

- Key idea: preserve local neighbors in 2D.
- Main formula: minimize $KL(P || Q)$.
- When to use: visualization.
- Metrics: visual quality, trustworthiness.
- Common traps: overinterpreting clusters and distances.
- Interview one-liner: t-SNE is a nonlinear local-neighborhood visualization method.

20. Final Cheat Sheet

Item	Summary
Definition	Nonlinear dimensionality reduction for visualization
Input/output	embeddings/features -> 2D coordinates
Main steps	compute similarities, optimize low-D map
Hyperparameters	perplexity, learning rate, iterations

Item	Summary
Metrics	trustworthiness, visual stability
Pros	excellent local visual separation
Cons	slow, stochastic, weak global meaning
Best use	embedding inspection

UMAP

1. Overview

UMAP, or Uniform Manifold Approximation and Projection, is a nonlinear dimensionality reduction method used for visualization and sometimes preprocessing. It often runs faster than t-SNE and can preserve more global structure.

2. Intuition

UMAP assumes high-dimensional data lies on a lower-dimensional manifold. It builds a neighbor graph in high dimensions, then tries to create a low-dimensional graph with similar neighbor relationships.

3. Prerequisites

- Nearest neighbors
- Graphs
- Distance metrics
- Embeddings
- Basic probability/objective intuition

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Manifold	Lower-dimensional structure inside high-D space	Basis of UMAP	Images vary by pose/lighting	Assumption
kNN graph	Graph of nearest neighbors	Captures local structure	Each point connected to 15 neighbors	n_neighbors
Fuzzy simplicial set	Weighted neighbor graph	Represents uncertainty	Strong/weak edges	High-level intuition enough
min_dist	Minimum spacing in embedding	Controls compactness	Tight vs spread clusters	Visualization tuning

Concept	Meaning	Why It Matters	Example	Interview Angle
Transform	Mapping new points	More practical than t-SNE	Embed new documents	Production angle

5. Algorithm / Working Process

Input: high-dimensional data.

Steps:

1. Find nearest neighbors.
2. Build weighted high-dimensional graph.
3. Initialize low-dimensional coordinates.
4. Optimize coordinates so low-dimensional graph resembles high-dimensional graph.
5. Return embedding.
6. Optionally transform new points using learned structure.

6. Mathematical Foundation

UMAP minimizes a fuzzy-set cross entropy between high-dimensional edge weights v_{ij} and low-dimensional edge weights w_{ij} :

$$C = \sum_{ij} [v_{ij} \log(v_{ij} / w_{ij}) + (1 - v_{ij}) \log((1 - v_{ij}) / (1 - w_{ij}))]$$

Low-dimensional similarity is modeled roughly as:

$$w_{ij} = 1 / (1 + a ||y_i - y_j||^{(2b)})$$

$n_neighbors$ controls local/global balance. min_dist controls how tightly points pack.

7. Practical Implementation

```
# pip install umap-learn
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
import umap

X, y = load_digits(return_X_y=True)
X = StandardScaler().fit_transform(X)

reducer = umap.UMAP(
    n_neighbors=15,
    min_dist=0.1,
    n_components=2,
    metric="euclidean",
    random_state=42
```

```

)
coords = reducer.fit_transform(X)

new_coords = reducer.transform(X[:3])
print(coords.shape)
print(new_coords)

```

8. Code Explanation

The code scales digit features and learns a 2D UMAP embedding. `n_neighbors=15` balances local detail and global structure. `transform` embeds new samples using the learned manifold approximation.

9. Training / Evaluation

Evaluate UMAP visually, with trustworthiness, downstream model performance, and stability across seeds. For supervised tasks, fit UMAP only on training data to avoid leakage.

10. Complexity and Cost

UMAP uses approximate nearest neighbors and is usually faster than t-SNE for large datasets. Cost depends heavily on neighbor search. CPU is enough for many datasets.

11. Common Use Cases

- Embedding visualization
- Single-cell biology
- Document maps
- Preprocessing before clustering
- Interactive data exploration

12. Common Mistakes

- Treating visualization distances as exact
- Not scaling numeric data
- Choosing `n_neighbors` without checking stability
- Using UMAP before train/test split
- Assuming clusters are real without validation
- Ignoring random seed effects

13. Edge Cases / Limitations

UMAP can distort structure, depends on parameters, and can produce different layouts across seeds. It may preserve misleading neighborhoods if the original distance metric is poor.

14. Variations

Variation	What Changes	When To Use	Importance
Supervised UMAP	Uses labels during embedding	Better class separation	Medium

Variation	What Changes	When To Use	Importance
Parametric UMAP	Neural network learns mapping	Large/new data	Research
densMAP	Preserves density better	Density-sensitive visualization	Medium
AlignedUMAP	Aligns multiple embeddings	Time-series manifolds	Low/Research

15. Related Topics

UMAP is often compared with t-SNE. It is usually faster, can transform new data, and may preserve global structure better. PCA is linear and often used before UMAP for denoising.

16. Interview Questions

1. What is UMAP used for? Nonlinear dimensionality reduction and visualization.
2. UMAP vs t-SNE? UMAP is often faster and can transform new data.
3. What does `n_neighbors` control? Local vs global structure balance.
4. What does `min_dist` control? Tightness of points in low-dimensional space.
5. Is UMAP supervised? Standard UMAP is unsupervised; supervised UMAP exists.
6. Does UMAP preserve exact distances? No.
7. Why scale data? Neighbor search depends on distance.
8. Can UMAP be used before clustering? Yes, but validate because it distorts data.
9. What graph does UMAP build? A weighted nearest-neighbor graph.
10. Main risk? Overinterpreting visual clusters.

17. Practice Tasks

- Install `umap-learn` and visualize digits.
- Compare `n_neighbors=5`, `15`, and `50`.
- Compare `min_dist=0.0` and `0.8`.
- Cluster UMAP output and evaluate carefully.
- Use UMAP on sentence embeddings.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Semantic Map	Interactive map of text embeddings	UMAP, Streamlit	News headlines	NLP visualization
Cell Type Explorer	Visualizes cell populations	scanpy, UMAP	PBMC	Research internship
Product Similarity Map	Shows similar products	sklearn, UMAP	Retail products	Recommender insight

19. Quick Revision

- Key idea: preserve neighbor graph in low dimensions.

- Main formula: fuzzy graph cross entropy.
- When to use: fast nonlinear visualization.
- Metrics: trustworthiness, stability.
- Common traps: leakage and overinterpretation.
- Interview one-liner: UMAP learns a low-dimensional layout that preserves nearest-neighbor structure.

20. Final Cheat Sheet

Item	Summary
Definition	Manifold-based dimensionality reduction
Input/output	high-D data -> low-D embedding
Main steps	kNN graph, low-D graph optimization
Hyperparameters	<code>n_neighbors</code> , <code>min_dist</code> , metric
Metrics	trustworthiness, visual/domain checks
Pros	fast, strong visualizations, transform support
Cons	parameter-sensitive
Best use	embedding exploration

Apriori

1. Overview

Apriori is a classic algorithm for mining frequent itemsets. It powers association rule mining by finding item combinations that occur often enough in transaction data.

2. Intuition

If `{bread, milk, butter}` is frequent, then `{bread, milk}`, `{bread, butter}`, and `{milk, butter}` must also be frequent. Apriori uses this fact to avoid checking impossible large itemsets.

3. Prerequisites

- Set operations
- Transaction encoding
- Support metric
- Combinatorics
- Basic pandas

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Apriori property	Subsets of frequent itemset are frequent	Prunes search	Infrequent bread+jam blocks larger sets	Core trick
Candidate generation	Create possible k -itemsets	Search step	Join frequent pairs	Complexity
Pruning	Remove candidates with infrequent subsets	Saves work	Drop impossible triples	Efficiency
Support counting	Count candidate frequency	Decides frequent sets	Scan baskets	Bottleneck
Rule generation	Convert itemsets to rules	Business output	$\{A, B\} \rightarrow \{C\}$	Separate from itemset mining

5. Algorithm / Working Process

Input: transactions and minimum support.

Steps:

1. Find frequent 1-itemsets.
2. Generate candidate 2-itemsets from frequent 1-itemsets.
3. Count candidate support.
4. Keep itemsets meeting minimum support.
5. Repeat for larger k .
6. Stop when no candidates remain.
7. Generate association rules from frequent itemsets.

6. Mathematical Foundation

Apriori property:

If itemset I is frequent, every subset S subset I is frequent.
 Contrapositive: if S is infrequent, every superset of S is infrequent.

Support:

$$\text{support}(I) = \text{number of transactions containing } I / \text{total transactions}$$

7. Practical Implementation

```
from itertools import combinations
```

```

transactions = [
    {"bread", "milk"},
    {"bread", "diaper", "beer", "egg"},
    {"milk", "diaper", "beer", "cola"},
    {"bread", "milk", "diaper", "beer"},
    {"bread", "milk", "diaper", "cola"},
]

def support(itemset):
    count = sum(itemset <= transaction for transaction in transactions)
    return count / len(transactions)

min_support = 0.4
items = sorted(set().union(*transactions))
frequent = []
current = [{item} for item in items if support({item}) >= min_support]

k = 1
while current:
    frequent.extend(current)
    candidates = []
    for a, b in combinations(current, 2):
        candidate = a | b
        if len(candidate) == k + 1 and candidate not in candidates:
            if all(set(sub) in current for sub in combinations(candidate, k)):
                candidates.append(candidate)
    current = [c for c in candidates if support(c) >= min_support]
    k += 1

for itemset in frequent:
    print(itemset, support(itemset))

```

8. Code Explanation

The code implements Apriori directly. It starts with frequent single items, joins them into larger candidates, prunes candidates whose subsets are not frequent, and keeps only itemsets above support threshold.

9. Training / Evaluation

Apriori has no train/test phase. Evaluate by support, number of discovered itemsets, rule quality after generation, interpretability, and business impact.

10. Complexity and Cost

Worst-case complexity is exponential in the number of items. Multiple database scans can be expensive. Apriori works best when minimum support is not too low and transactions are not extremely dense.

11. Common Use Cases

- Frequent product bundle discovery
- Feature co-occurrence mining

- Rule-based recommendations
- Medical code pattern mining
- Web event co-occurrence

12. Common Mistakes

- Setting minimum support too low
- Confusing frequent itemsets with association rules
- Not deduplicating items inside transactions
- Ignoring rare but important items
- Running Apriori on too many unique items
- Using it for ordered sequence patterns

13. Edge Cases / Limitations

Apriori is slow for dense datasets, low support thresholds, and large item vocabularies. It ignores order, quantity, user identity, and causality.

14. Variations

Variation	What Changes	When To Use	Importance
FP-Growth	Avoids candidate explosion with FP-tree	Large itemsets	High
Eclat	Uses vertical transaction IDs	Dense data	Medium
Hash-based Apriori	Hashes candidates	Speed improvement	Low
Dynamic Itemset Counting	Adds candidates during scans	Fewer passes	Low

15. Related Topics

Apriori is the frequent itemset mining step behind association rule mining. FP-Growth is a faster alternative. Matrix factorization solves recommendation through latent factors instead of explicit item rules.

16. Interview Questions

1. What is Apriori used for? Frequent itemset mining.
2. What is the Apriori property? All subsets of a frequent itemset are frequent.
3. Why is pruning valid? Infrequent subsets imply infrequent supersets.
4. What is minimum support? Frequency threshold for itemsets.
5. Apriori vs association rules? Apriori finds itemsets; rules are generated afterward.
6. Why can Apriori be slow? Candidate explosion and repeated scans.
7. What happens if support is too low? Too many candidates and noisy rules.
8. Does Apriori handle item order? No.
9. Alternative to Apriori? FP-Growth.
10. Is Apriori supervised? No.

17. Practice Tasks

- Implement support counting from scratch.
- Mine frequent itemsets from grocery data.
- Vary minimum support and plot itemset count.
- Generate rules from frequent triples.
- Compare Apriori with FP-Growth.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Frequent Basket Miner	Finds popular item bundles	Python, pandas	Instacart	Retail ML
Course Combo Analyzer	Finds courses students take together	pandas	University enrollment	EdTech analytics
Diagnosis Pattern Miner	Finds frequent symptom sets	pandas	Medical records demo	Healthcare analytics

19. Quick Revision

- Key idea: prune supersets of infrequent itemsets.
- Main formula: $\text{support}(I) = \text{count}(I) / N$.
- When to use: frequent itemset mining.
- Metrics: support and later rule metrics.
- Common traps: low support, huge item vocab.
- Interview one-liner: Apriori mines frequent itemsets using subset-based pruning.

20. Final Cheat Sheet

Item	Summary
Definition	Frequent itemset mining algorithm
Input/output	transactions -> frequent itemsets
Main steps	generate candidates, count support, prune
Hyperparameters	min support
Metrics	support, itemset count
Pros	simple, interpretable
Cons	exponential worst case
Best use	small/medium basket mining

Isolation Forest

1. Overview

Isolation Forest is an unsupervised anomaly detection algorithm based on the idea that anomalies are easier to isolate than normal points. It builds random trees and scores points by average path length.

It is a top interview algorithm for tabular anomaly detection because it is fast, scalable, and works without labels.

2. Intuition

In a dataset of normal transactions, most points are packed in dense regions. Random splits need many cuts to isolate one normal point. An outlier sitting far away can be isolated with very few random cuts.

3. Prerequisites

- Decision trees
- Random sampling
- Path length
- Anomaly detection metrics
- Feature preprocessing

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Isolation	Separating a point from others	Core anomaly idea	Far point split quickly	Different from density
Random split	Random feature and threshold	Creates isolation tree	Split age at 43.2	No supervised impurity
Path length	Splits needed to isolate point	Anomaly signal	Short path = anomaly	Main scoring idea
Subsampling	Train each tree on sample	Improves speed and anomaly contrast	256 samples/tree	Scalability
Contamination	Expected anomaly proportion	Threshold labels	1% alerts	Score vs decision

5. Algorithm / Working Process

Input: feature matrix X .

Training:

1. For each tree, sample a subset of rows.
2. Randomly select a feature.
3. Randomly choose a split between min and max feature value.
4. Recursively split until point is isolated, max depth is reached, or node has one value.
5. Repeat for many trees.

Inference:

1. Pass point through all trees.
2. Compute average path length.
3. Convert path length to anomaly score.
4. Threshold score if labels are needed.

6. Mathematical Foundation

Average path length:

$$E(h(x)) = \text{average path length of } x \text{ across trees}$$

Normalization constant for sample size ψ :

$$c(\psi) = 2 H(\psi - 1) - 2(\psi - 1)/\psi$$

where $H(i)$ is the harmonic number.

Anomaly score:

$$s(x, \psi) = 2^{-E(h(x)) / c(\psi)}$$

Interpretation:

- s near 1: likely anomaly
- s around 0.5: uncertain
- s much less than 0.5: likely normal

7. Practical Implementation

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.metrics import classification_report, average_precision_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(42)
normal = rng.normal(0, 1, size=(1000, 4))
anomaly = rng.normal(5, 1, size=(40, 4))
X = np.vstack([normal, anomaly])
y = np.array([0] * len(normal) + [1] * len(anomaly))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
```

```

)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = IsolationForest(
    n_estimators=200,
    max_samples=256,
    contamination=0.04,
    random_state=42
)
model.fit(X_train)

scores = -model.score_samples(X_test)
pred = (model.predict(X_test) == -1).astype(int)

print("PR-AUC:", average_precision_score(y_test, scores))
print(classification_report(y_test, pred))

```

8. Code Explanation

The training set contains mostly normal data. Scaling is used because mixed feature scales can affect random threshold behavior. `score_samples` is negated so larger values mean more anomalous. PR-AUC is better than accuracy for rare anomalies.

9. Training / Evaluation

Use labels if available only for evaluation/thresholding, not model fitting. Metrics: precision, recall, F1, PR-AUC, ROC-AUC, alert rate, and investigation cost.

Important hyperparameters:

Hyperparameter	Meaning	Practical Tip
<code>n_estimators</code>	Number of trees	More trees improve stability
<code>max_samples</code>	Subsample size per tree	256 is common default
<code>contamination</code>	Expected anomaly ratio	Sets decision threshold
<code>max_features</code>	Feature subsampling	Useful with many noisy features

10. Complexity and Cost

Training is roughly $O(t * \text{psi} * \log \text{psi})$ where `t` is trees and `psi` is subsample size. Inference is $O(t * \text{tree_depth})$. It is CPU-friendly and scales well.

11. Common Use Cases

- Fraud detection baseline
- Cybersecurity event detection

- Data quality monitoring
- Sensor fault detection
- Outlier filtering before modeling
- Transaction alert ranking

12. Common Mistakes

- Evaluating with accuracy
- Using labels during unsupervised fitting incorrectly
- Setting contamination equal to a guess without validating alert volume
- Forgetting time-based validation
- Not checking feature drift
- Assuming all rare points are bad
- Ignoring categorical encoding quality

13. Edge Cases / Limitations

Isolation Forest struggles when anomalies are not easier to isolate, when anomalies form dense groups, when features are irrelevant/noisy, or when normal data is highly multi-modal.

14. Variations

Variation	What Changes	When To Use	Importance
Extended Isolation Forest	Random hyperplane splits	Axis-aligned splits fail	Medium
SCiForest	Uses selected split criteria	Better anomaly separation	Low
Streaming variants	Updates over time	Real-time monitoring	Medium
Feature bagging IF	Subsample features	High-dimensional data	Medium

15. Related Topics

Isolation Forest is related to Random Forest only structurally; it does not use labels or impurity. It compares with One-class SVM as a faster tabular baseline. It compares with LOF because LOF uses local density while Isolation Forest uses random partition depth.

16. Interview Questions

1. Why do anomalies have shorter paths? They are rare and different, so random splits isolate them quickly.
2. Is Isolation Forest supervised? No, labels are not required.
3. What does contamination do? Sets threshold for converting scores to anomaly labels.
4. What is `max_samples`? Number of samples used to build each tree.
5. Why subsample? It improves speed and makes anomalies stand out.
6. Does it use Gini or entropy? No, splits are random.
7. Which metric for rare anomalies? PR-AUC, precision, recall, and F1.
8. What does `predict` return in sklearn? `-1` for anomalies and `1` for normal.
9. When does it fail? Dense anomaly clusters or poor features.

10. Isolation Forest vs One-class SVM? Isolation Forest scales better and needs less kernel tuning.

17. Practice Tasks

- Build Isolation Forest on synthetic anomalies.
- Tune contamination for a target alert rate.
- Compare PR-AUC with ROC-AUC.
- Plot anomaly scores over time.
- Test effect of noisy irrelevant features.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Fraud Triage System	Ranks risky transactions	sklearn, FastAPI	Kaggle fraud	Production-style anomaly ML
Sensor Drift Monitor	Detects abnormal machine states	pandas, sklearn	NASA turbofan	Industrial AI
Data Quality Sentinel	Flags weird rows in pipelines	sklearn, Great Expectations	Any tabular data	MLOps relevance

19. Quick Revision

- Key idea: anomalies are isolated in fewer random splits.
- Main formula: $s(x) = 2^{(-E(h(x))/c(psi))}$.
- When to use: tabular anomaly detection baseline.
- Metrics: PR-AUC, precision, recall.
- Common traps: accuracy and bad contamination.
- Interview one-liner: Isolation Forest detects anomalies by measuring how quickly random trees isolate each point.

20. Final Cheat Sheet

Item	Summary
Definition	Random-tree anomaly detector
Input/output	features -> anomaly score/label
Main steps	random trees, path length, score
Hyperparameters	trees, max samples, contamination
Metrics	PR-AUC, precision, recall
Pros	fast, scalable, no labels
Cons	weak if anomalies are not easily isolated
Best use	tabular anomaly baseline

One-Class SVM

1. Overview

One-class SVM is an unsupervised or semi-supervised anomaly detection method that learns a boundary around normal data. Points outside the boundary are considered anomalies.

It is useful for novelty detection when training data is mostly clean and normal.

2. Intuition

Imagine drawing a flexible fence around normal examples. Future points inside the fence are accepted as normal; points outside are rejected as anomalies.

3. Prerequisites

- SVM margin intuition
- Kernels
- Feature scaling
- Optimization basics
- Anomaly metrics

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
Support vectors	Boundary-defining points	Define decision frontier	Edge normal samples	SVM connection
Kernel	Similarity function	Nonlinear boundary	RBF kernel	Gamma tuning
ν	Upper bound on outlier fraction, lower bound on support vectors	Controls strictness	$\nu=0.05$	Common question
Decision function	Signed distance to boundary	Anomaly score	Negative outside	Thresholding
Novelty detection	Train on normal data only	Clean setting	Manufacturing normal samples	Difference from outlier detection

5. Algorithm / Working Process

Input: mostly normal data.

Training:

1. Scale features.

2. Choose kernel, often RBF.
3. Learn boundary that separates data from origin in feature space.
4. Allow some violations controlled by ν .

Inference:

1. Compute decision score.
2. Positive score means normal.
3. Negative score means anomaly.

6. Mathematical Foundation

One-class SVM solves:

$$\min_{\{w, \rho, \xi\}} \frac{1}{2} \|w\|^2 + \frac{1}{(\nu n)} \sum_i \xi_i - \rho$$

subject to:

$$w \cdot \phi(x_i) \geq \rho - \xi_i$$

$$\xi_i \geq 0$$

Decision function:

$$f(x) = \text{sign}(w \cdot \phi(x) - \rho)$$

RBF kernel:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

7. Practical Implementation

```
import numpy as np
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

rng = np.random.default_rng(42)
X_train = rng.normal(0, 1, size=(400, 2)) # mostly normal
X_test_normal = rng.normal(0, 1, size=(100, 2))
X_test_anom = rng.normal(4, 1, size=(20, 2))
X_test = np.vstack([X_test_normal, X_test_anom])
y_test = np.array([0] * 100 + [1] * 20)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = OneClassSVM(kernel="rbf", gamma="scale", nu=0.05)
model.fit(X_train)

pred = (model.predict(X_test) == -1).astype(int)
scores = -model.decision_function(X_test)

print(classification_report(y_test, pred))
print("Top anomaly scores:", np.sort(scores.ravel())[-5:])
```

8. Code Explanation

The model trains only on normal-looking data. `nu=0.05` allows roughly 5% training violations. `predict` returns `-1` for anomalies. The negative decision function is used as an anomaly score where larger means more suspicious.

9. Training / Evaluation

Train on clean normal data when possible. Validate using labeled anomalies, simulated anomalies, or expert review. Metrics include precision, recall, F1, PR-AUC, false positive rate, and alert volume.

10. Complexity and Cost

Kernel One-class SVM can be expensive, often between $O(n^2)$ and $O(n^3)$ training depending on solver/data. It is not ideal for very large datasets. Linear variants scale better.

11. Common Use Cases

- Manufacturing defect detection
- Novelty detection
- Intrusion detection
- Medical abnormality screening
- Small/medium tabular anomaly detection

12. Common Mistakes

- Not scaling features
- Training on contaminated data
- Misunderstanding `nu`
- Using RBF SVM on huge datasets
- Not tuning `gamma`
- Reporting accuracy on imbalanced data
- Confusing outlier detection and novelty detection

13. Edge Cases / Limitations

One-class SVM is sensitive to scaling, `gamma`, and `nu`. It can be slow on large data and brittle when normal data has many modes.

14. Variations

Variation	What Changes	When To Use	Importance
RBF One-class SVM	Nonlinear boundary	Small/medium nonlinear normal region	High
Linear One-class SVM	Linear boundary	Large sparse data	Medium
SGD One-class SVM	Approximate scalable training	Large data	Medium
SVDD	Minimum enclosing hypersphere	Similar novelty detection	Research

15. Related Topics

One-class SVM relates to standard SVM but uses only one class. It competes with Isolation Forest, LOF, and autoencoder anomaly detection. Isolation Forest is usually easier to scale.

16. Interview Questions

1. What is One-class SVM? A model that learns a boundary around normal data.
2. Is it supervised? Usually unsupervised/semi-supervised with normal-only data.
3. What does `nu` mean? It controls allowed outlier fraction and support vector lower bound.
4. Why use RBF kernel? To learn nonlinear boundaries.
5. Why scale data? SVM kernels depend on distances.
6. What does negative prediction mean in sklearn? Anomaly.
7. One-class SVM vs Isolation Forest? SVM learns a kernel boundary; Isolation Forest uses random isolation and scales better.
8. When avoid One-class SVM? Very large datasets or poorly scaled noisy features.
9. What is novelty detection? Detecting unseen abnormal data after training on normal data.
10. How tune threshold? Use validation anomalies, cost, or alert budget.

17. Practice Tasks

- Train One-class SVM on normal synthetic data.
- Plot decision boundary in 2D.
- Tune `nu` and `gamma`.
- Compare with Isolation Forest.
- Evaluate with PR-AUC on imbalanced data.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Defect Novelty Detector	Flags abnormal product measurements	sklearn	UCI quality data	Manufacturing ML

Project	What It Does	Tech Stack	Dataset	Resume Value
Login Behavior Detector	Finds unusual login sessions	pandas, sklearn	Auth logs	Security analytics
Medical Screening Baseline	Flags abnormal patient records	sklearn	UCI medical datasets	Healthcare ML

19. Quick Revision

- Key idea: learn boundary around normal data.
- Main formula: maximize separation from origin in feature space.
- When to use: normal-only novelty detection.
- Metrics: precision, recall, PR-AUC.
- Common traps: no scaling, bad ν/γ .
- Interview one-liner: One-class SVM detects anomalies by learning the support boundary of normal data.

20. Final Cheat Sheet

Item	Summary
Definition	SVM-based novelty/anomaly detector
Input/output	normal training data -> normal/anomaly decision
Main steps	scale, fit boundary, score new points
Hyperparameters	kernel, ν , γ
Metrics	precision, recall, F1, PR-AUC
Pros	strong nonlinear boundary for small data
Cons	slow, sensitive to scaling/tuning
Best use	clean normal-only novelty detection

Matrix Factorization

1. Overview

Matrix factorization decomposes a large matrix into smaller latent-factor matrices. In ML interviews it is most often discussed for recommender systems, where a user-item rating matrix is factorized into user and item embeddings.

2. Intuition

A movie rating matrix may be explained by hidden factors like action preference, romance preference, and comedy preference. Each user and movie gets coordinates in this hidden factor space. The dot product predicts ratings.

3. Prerequisites

- Linear algebra
- Dot product
- Gradient descent
- Regularization
- Sparse matrices
- Recommender metrics

4. Core Concepts

Concept	Meaning	Why It Matters	Example	Interview Angle
User factors	User embedding vector	Captures preferences	User likes action	Latent features
Item factors	Item embedding vector	Captures item attributes	Movie is action-heavy	Dot-product prediction
Sparse matrix	Most ratings missing	Real recommender challenge	User rated 20 of 10k movies	Missing not equal zero
Reconstruction	Approximate original ratings	Prediction goal	$R \approx U V^T$	Low-rank assumption
Regularization	Penalizes large factors	Prevents overfitting	L2 penalty	Cold-start and sparsity

5. Algorithm / Working Process

Input: observed user-item interactions.

Steps:

1. Create sparse rating matrix R .
2. Initialize user matrix P and item matrix Q .
3. Predict rating with dot product.
4. Compute loss only on observed entries.
5. Update factors with SGD/ALS.
6. Recommend items with highest predicted scores.

6. Mathematical Foundation

Prediction:

$$\hat{r}_{ui} = p_u^T q_i$$

With biases:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$$

Objective:

$$\min \sum_{(u,i) \text{ in observed}} (r_{ui} - p_u^T q_i)^2 + \lambda (||p_u||^2 + ||q_i||^2)$$

Implicit feedback often uses confidence-weighted loss:

$$\min \sum_{u,i} c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda(\dots)$$

7. Practical Implementation

```
import numpy as np

ratings = np.array([
    [5, 4, 0, 0],
    [4, 0, 0, 1],
    [1, 0, 5, 4],
    [0, 1, 4, 5],
], dtype=float)

n_users, n_items = ratings.shape
k = 2
lr = 0.01
reg = 0.02
rng = np.random.default_rng(42)
P = rng.normal(0, 0.1, size=(n_users, k))
Q = rng.normal(0, 0.1, size=(n_items, k))

observed = np.argwhere(ratings > 0)

for _ in range(1000):
    for u, i in observed:
        err = ratings[u, i] - P[u] @ Q[i]
        P[u] += lr * (err * Q[i] - reg * P[u])
        Q[i] += lr * (err * P[u] - reg * Q[i])

predicted = P @ Q.T
print(np.round(predicted, 2))
```

8. Code Explanation

Zeros represent missing ratings, not real zero ratings. The loop updates only observed entries. $P[u] @ Q[i]$ predicts a rating from user and item latent vectors.

9. Training / Evaluation

Use train/test split over observed interactions, not matrix cells randomly without care. Explicit ratings use RMSE/MAE. Ranking recommenders use Precision@K, Recall@K, MAP@K, NDCG@K, and hit rate.

10. Complexity and Cost

SGD cost per epoch is $O(|\text{observed}| * k)$. Memory is $O((\text{users} + \text{items}) * k)$. It scales well to sparse data. GPU helps for neural recommenders but is not required for basic factorization.

11. Common Use Cases

- Movie recommendation
- Product recommendation
- Music recommendation
- Collaborative filtering
- Embedding users/items
- Missing value approximation

12. Common Mistakes

- Treating missing entries as zero ratings
- Randomly splitting after leakage through users/items
- Evaluating ranking with RMSE only
- Ignoring cold-start users/items
- Not using regularization
- Recommending already-consumed items

13. Edge Cases / Limitations

Matrix factorization struggles with cold start, changing preferences, sparse users, popularity bias, and lack of side information unless extended.

14. Variations

Variation	What Changes	When To Use	Importance
SVD	Matrix decomposition	Dense/filled matrices	Medium
FunkSVD	SGD on observed ratings	Recommenders	High
ALS	Alternating least squares	Large sparse data	High
NMF	Non-negative factors	Interpretability	Medium
Neural CF	Neural interaction function	Complex recommenders	Medium

15. Related Topics

Matrix factorization relates to PCA because both are low-rank approximations. It differs from association rules because it learns latent preferences rather than explicit item co-occurrence rules.

16. Interview Questions

1. What is matrix factorization? Decomposing a matrix into lower-rank factor matrices.
2. How used in recommendations? Factor user-item matrix into user and item embeddings.
3. What is predicted rating? Dot product of user and item vectors.
4. Why not treat missing as zero? Missing means unknown, not dislike.
5. What is cold start? New users/items lack interactions.
6. How evaluate recommendations? Ranking metrics like Recall@K and NDCG@K.
7. What does regularization do? Prevents overfitting latent factors.
8. ALS vs SGD? ALS solves alternating least squares; SGD updates observed entries.
9. What are latent factors? Hidden dimensions explaining preferences.
10. How add side information? Hybrid models with content/user features.

17. Practice Tasks

- Implement SGD matrix factorization.
- Add user and item biases.
- Evaluate RMSE on held-out ratings.
- Build top-10 recommendation list.
- Compare with item-item collaborative filtering.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Movie Recommender	Predicts and ranks movies	NumPy, pandas	MovieLens	Classic ML project
Product Recommender	Suggests products from purchases	implicit/sklearn	Instacart	E-commerce relevance
Music Taste Embeddings	Finds user/song factors	Python	Last.fm	Recommendation depth

19. Quick Revision

- Key idea: approximate R as $P Q^T$.
- Main formula: $\hat{r}_{ui} = p_u^T q_i$.
- When to use: collaborative filtering.
- Metrics: RMSE, Recall@K, NDCG@K.
- Common traps: missing-as-zero, cold start.
- Interview one-liner: matrix factorization learns user and item embeddings from sparse interactions.

20. Final Cheat Sheet

Item	Summary
Definition	Low-rank decomposition
Input/output	user-item matrix -> latent factors
Main steps	initialize, predict, optimize observed loss
Hyperparameters	rank, learning rate, regularization
Metrics	RMSE, Recall@K, NDCG@K
Pros	scalable, strong recommender baseline
Cons	cold start, sparsity
Best use	collaborative filtering